



Master's Thesis

Investigating a Hybrid LLM-GNN Model to Enhance the Efficiency of ADAPT-QAOA for Quantum Circuit Optimization

Supervisor: Assoc. Prof. Dr. Thoại Nam

Mai Chi Bao - 2370691

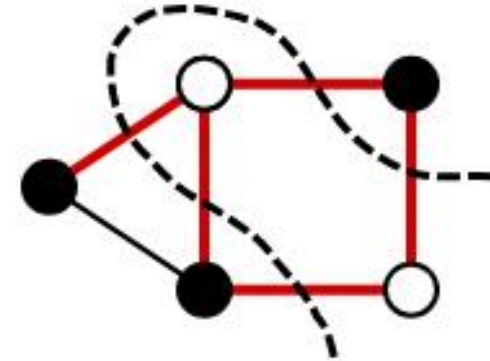
VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY

Problem Statement

Classical algorithms struggle with

Scalability & Computation Time

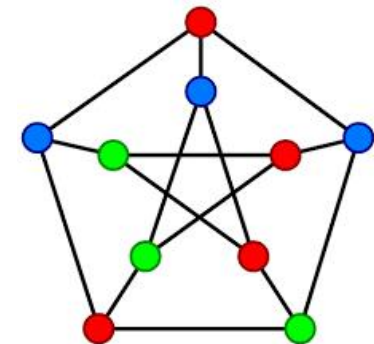
Max-Cut



Knapsack



Graph Coloring



How the system
changes with time

QAOA

State at time t

Random state

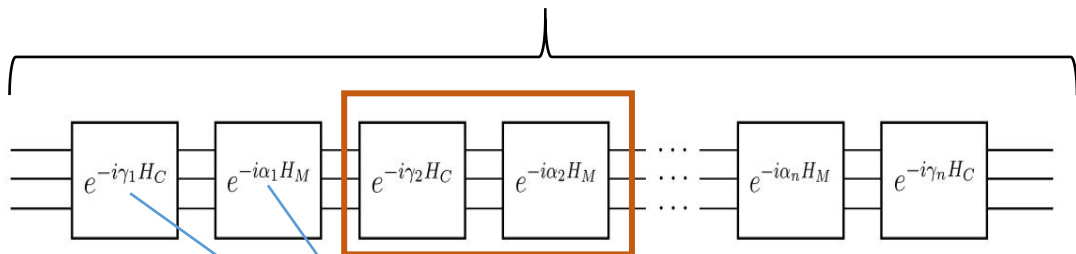
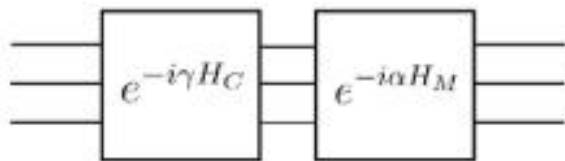
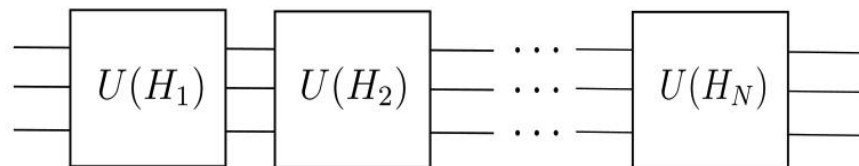
$$|\psi(t)\rangle$$

$$U(t)$$

$$|\psi(0)\rangle$$

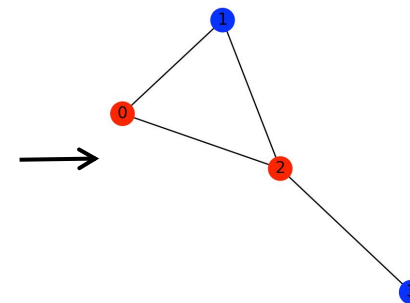
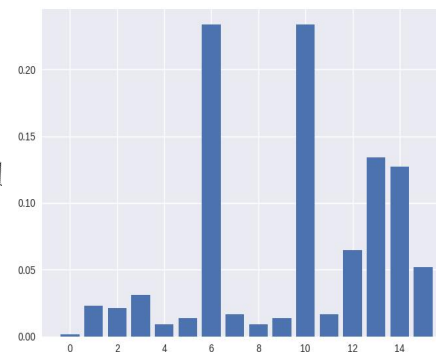
$$U = e^{-it\hat{H}}$$

$$H = H_1 + H_2 + \dots + H_N$$



Initial Parameters: γ, β

Run & Measure

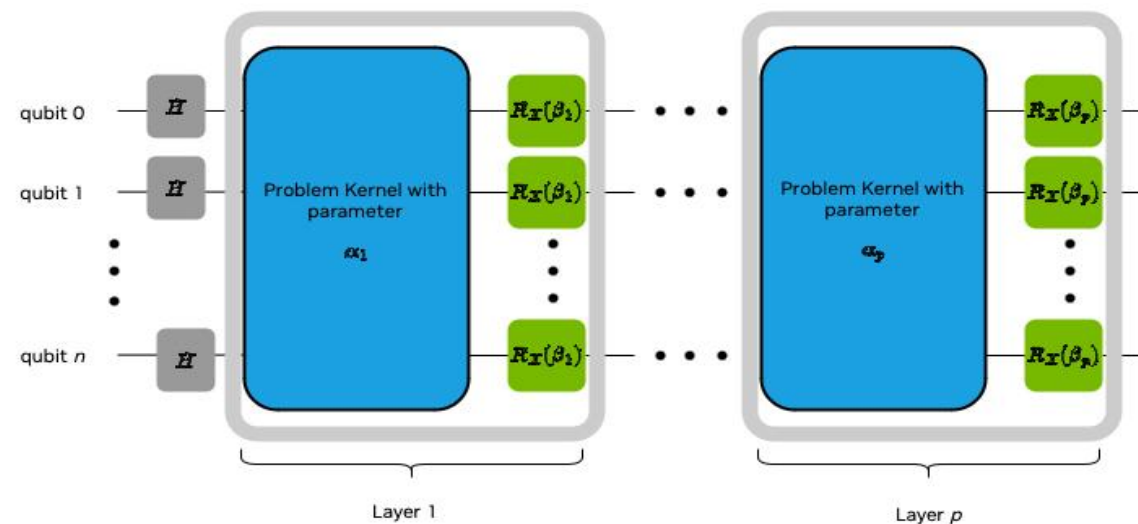


Problem Statement

Quantum computing (QAOA) offers a promising hybrid solution

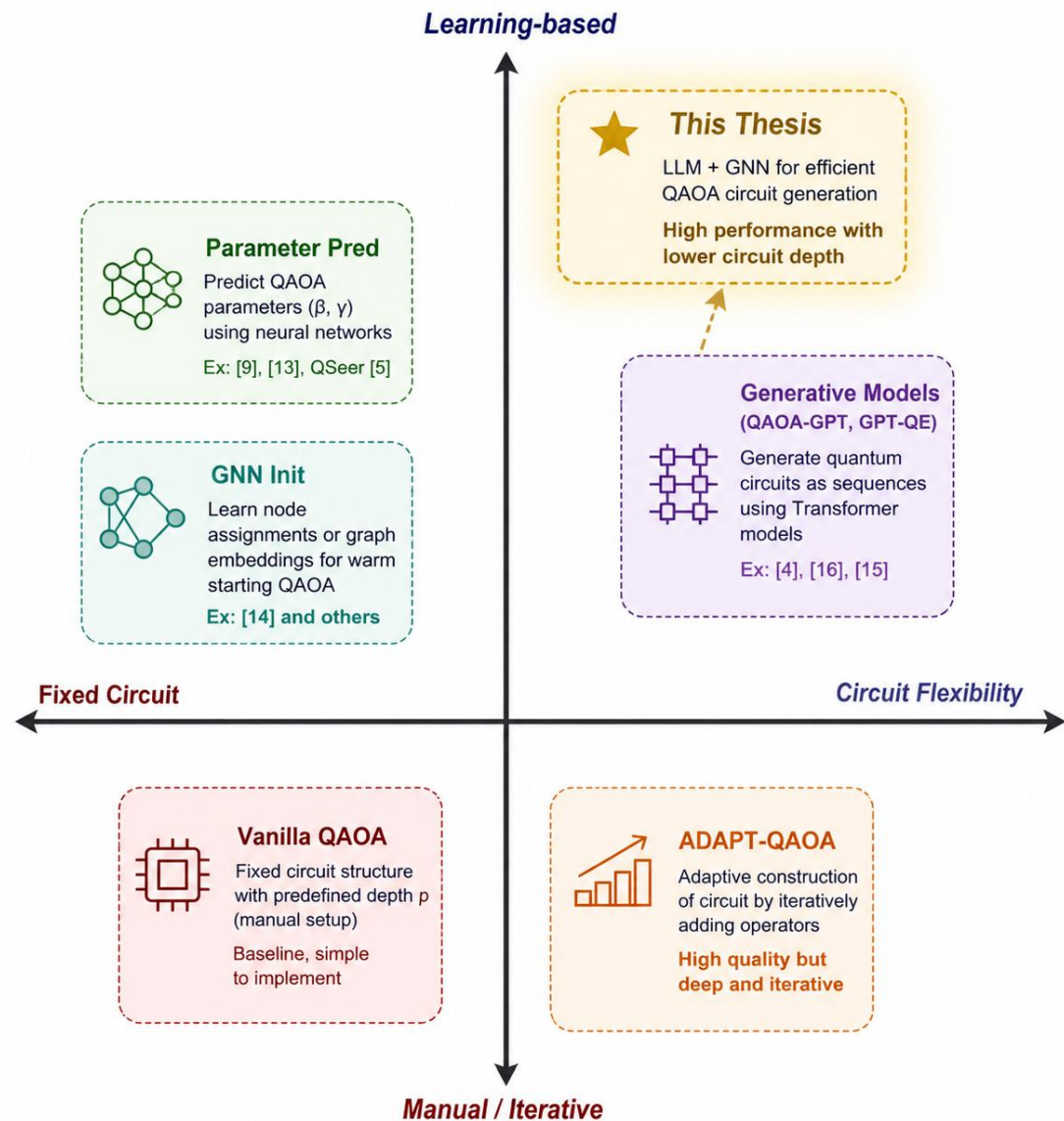
However, QAOA faces key challenges [1][2][7]:

- **Parameter initialization:** Poor initialization $\rightarrow$ slow convergence / barren plateaus
- **Circuit generation:** Exponential search space, lacks scalable methods
- **Generalization:** Limited performance across different depths & graph types



Related Work

- ADAPT-QAOA [7] circuit generation is largely unexplored, as it requires learning both operator selection and circuit structure dynamically, increasing complexity.
- Most learning-based methods [9][13][5] only predict QAOA parameters (β, γ), not the circuit structure, limiting them to fixed ansatz designs.
- Generative approaches (e.g., QAOA-GPT [4], GQE [16]) treat the task as sequence generation, but underutilize graph structural information, hurting generalization.

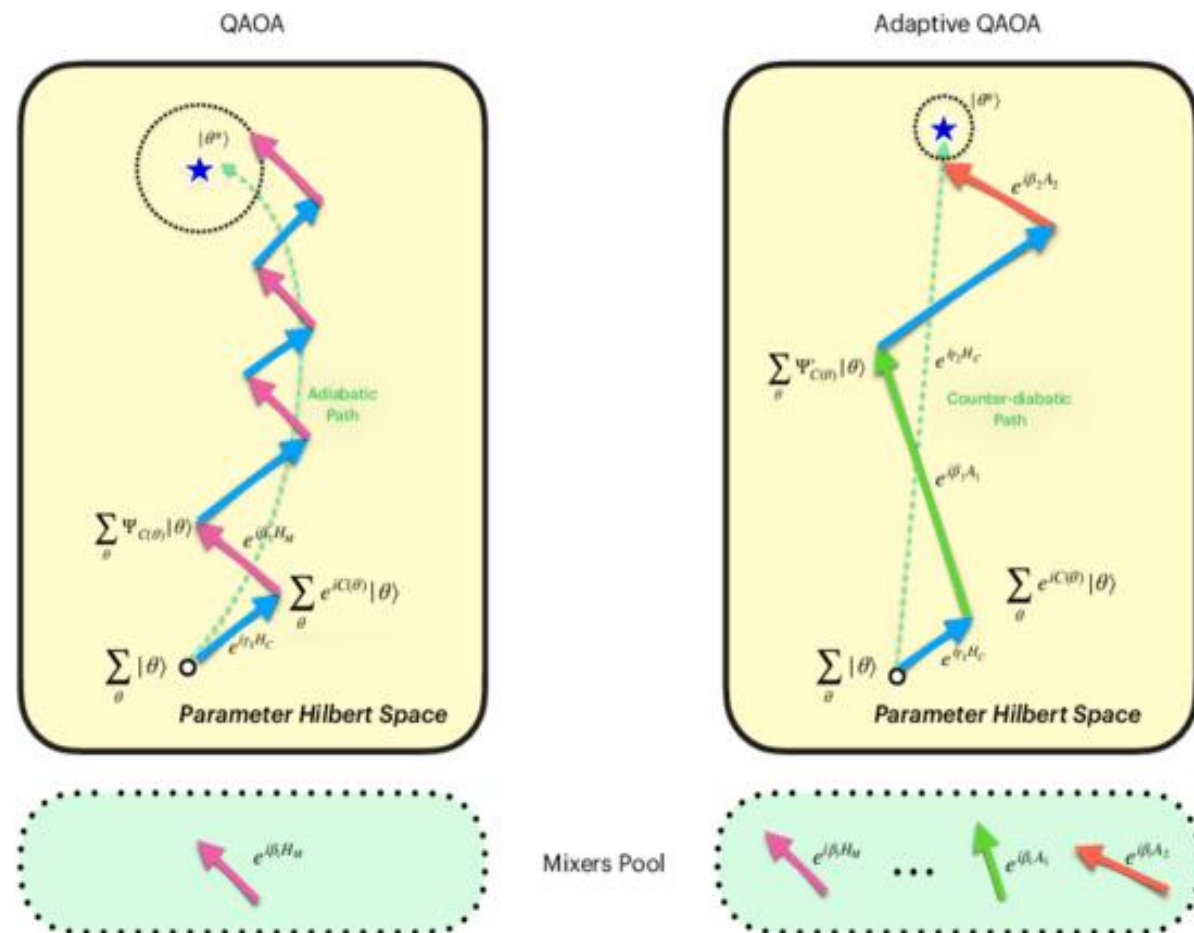


Preliminaries

ADAPT - QAOA

Advantages [7]

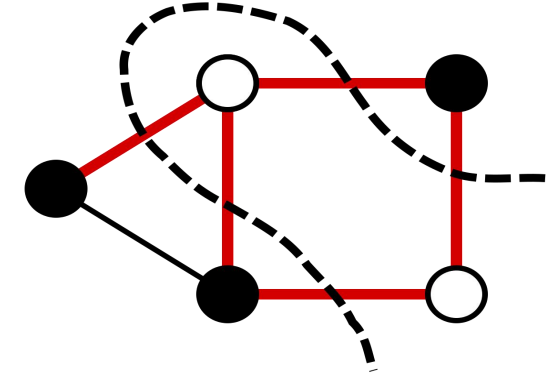
- Accelerates convergence thanks to an ansatz tailored for each problem.
- Reduces classical optimization cost - the number of parameters increases selectively, not fixed per layer.
- More effective for complex problems, where traditional QAOA requires large p to achieve good results.
- Flexible and well-suited for problems with specific structures (such as Max-Cut, weighted graphs).



QAOA for Max-Cut Problem

Max-cut Problem

- Partition the graph $G = (V, E)$ into 2 sets such that the number of edges cut between the two sides is maximized.
- A typical combinatorial optimization problem $\rightarrow$ well-suited for QAOA.



Encode into Quantum Form [8]

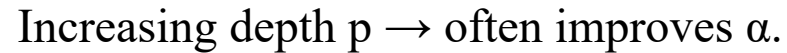
$$H_z = \frac{1}{2} \sum_{(j,k) \in E} w_{j,k} (1 - Z_j Z_k)$$

- Each node $\rightarrow$ 1 Qubit
- Each edge $\rightarrow$ interaction between 2 qubits
- Output: quantum state $\rightarrow$ measure $\rightarrow$ candidate solution
- Total parameters: $2p$ (γ, β)
- Measures how “good” a cut is

Expected Energy

$0 \leq \alpha \leq 1 \rightarrow \alpha \equiv \frac{F(\gamma^*, \beta^*)}{C_{\max}}$

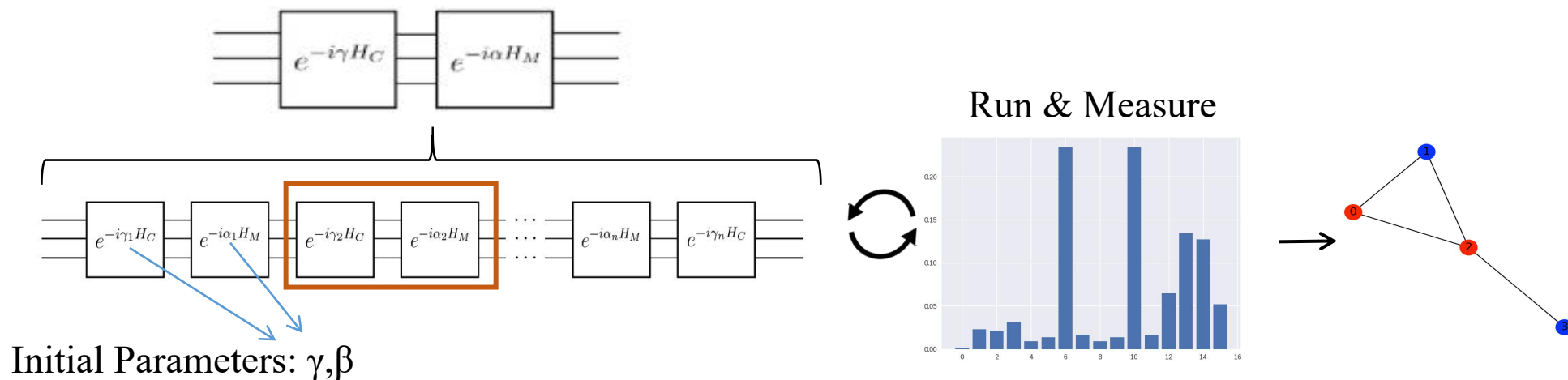
Maximum possible Max-Cut value



Why not increase p

- Expected Energy
- $0 \leq \alpha \leq 1 \rightarrow \alpha \equiv \frac{F(\gamma^*, \beta^*)}{C_{\max}}$
- Maximum possible Max-Cut value

QAOA for Max-Cut Problem



Good initialization of γ, β is needed:

- Helps avoid barren plateaus
- Guides optimization into regions with useful gradients
- Accelerates convergence and improves Max-Cut solutions
- Especially important when p is large

Expected Energy

$$0 \leq \alpha \leq 1 \rightarrow \alpha \equiv \frac{F(\gamma^*, \beta^*)}{C_{\max}}$$

Maximum possible Max-Cut value

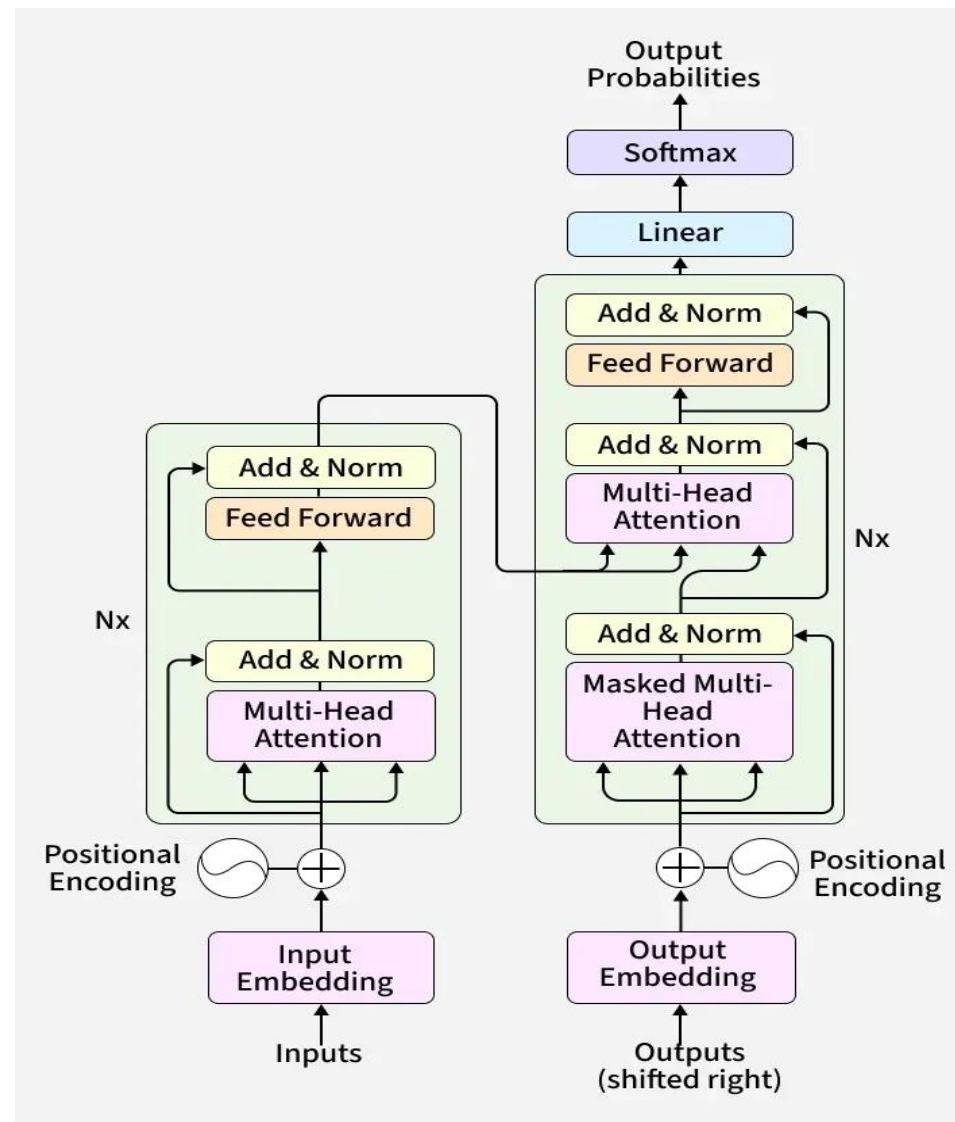
Large Language Models

Autoregressive next-token prediction loss

$$U = \{u_1, u_2, \dots, u_n\} \xrightarrow{\text{Maximize}} \mathcal{L}_{LM}(U) = \sum_{i=1}^n \log P(u_i | u_1, u_2, \dots, u_{i-1}; \theta)$$

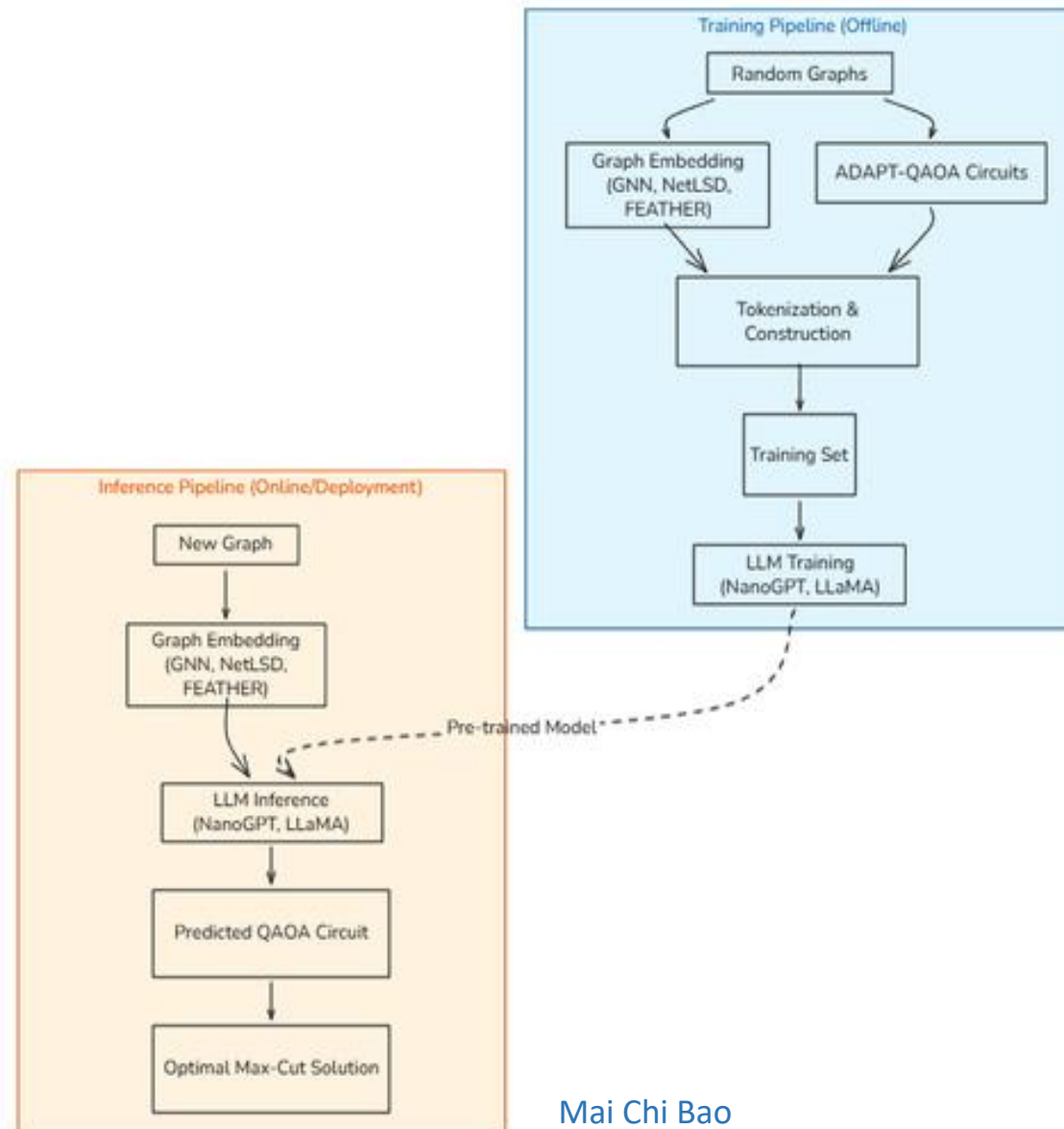
Why LLMs:

- Learn patterns from large datasets to generate & optimize quantum circuits.
- Evaluate generalization ability and explore circuit structures more effectively.
- Serve as a foundation for the generative modeling approach in designing adaptive QAOA circuits.

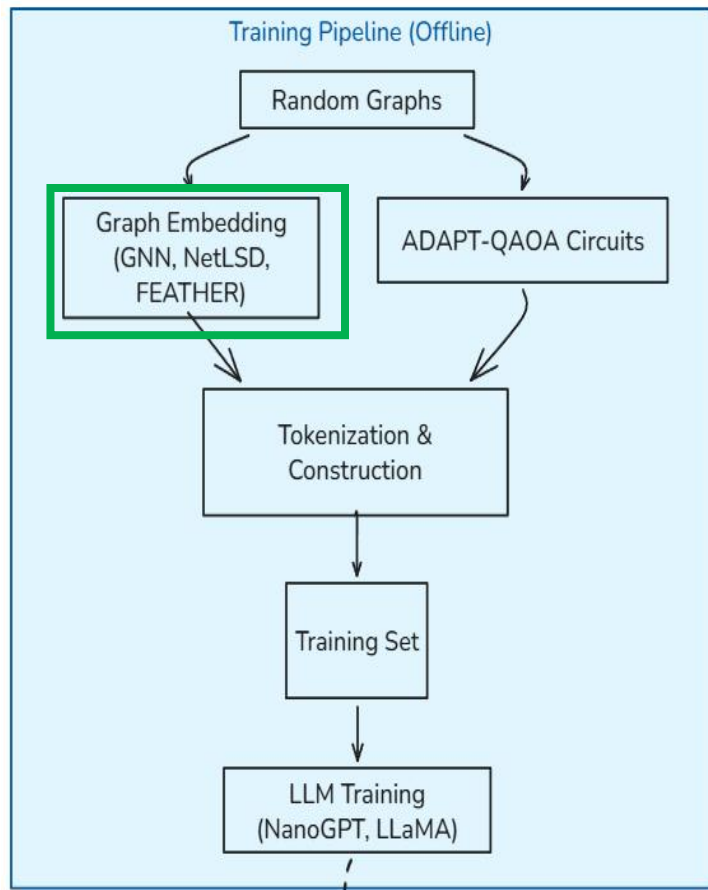


Methodology

Hybrid LLM - GNN Assisted ADAPT-QAOA Framework



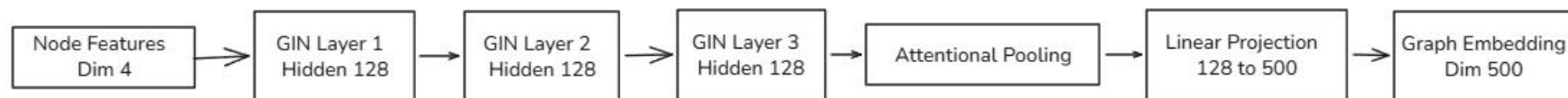
Graph Embeddings



Criteria	NetLSD [11]	Feather [10]	GNNs [6]
Definition	Encodes a graph by analyzing heat diffusion over the Laplacian eigenvalue spectrum across multiple time scales to capture both local and global structure.	Builds graph embeddings by transforming node features (via sin/cos) and propagating them through the normalized adjacency matrix to capture multi-hop structural patterns.	Learns graph representations through iterative message passing, where node features are aggregated from neighbors and pooled into a graph-level embedding.
Need Training	No	No	Yes (end-to-end)
Structural Focus	Mainly Global (multi-scale Laplacian spectrum)	Global + Local (spectral + attributes)	Local → Global (multi-hop message passing)
Supports Node Attributes	No	Yes	Yes
Supports Edge Attributes	No	Limited	Yes
Computational Speed & Scalability	Fast & scalable	Fast & scalable	Slower & Depends on model depth and graph size
Optimization Capability	Fixed embedding (no task adaptation)	Fixed embedding	Task-adaptive representation

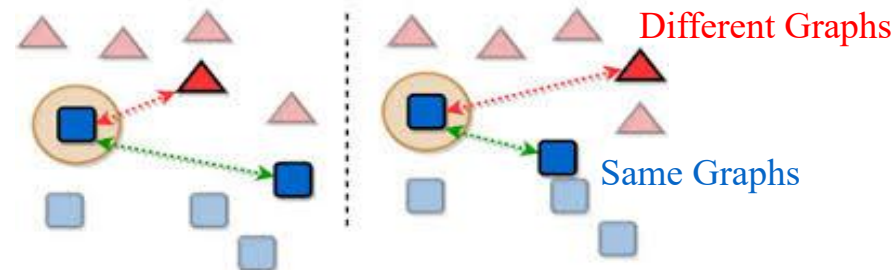
Graph Embeddings

GNN Architecture



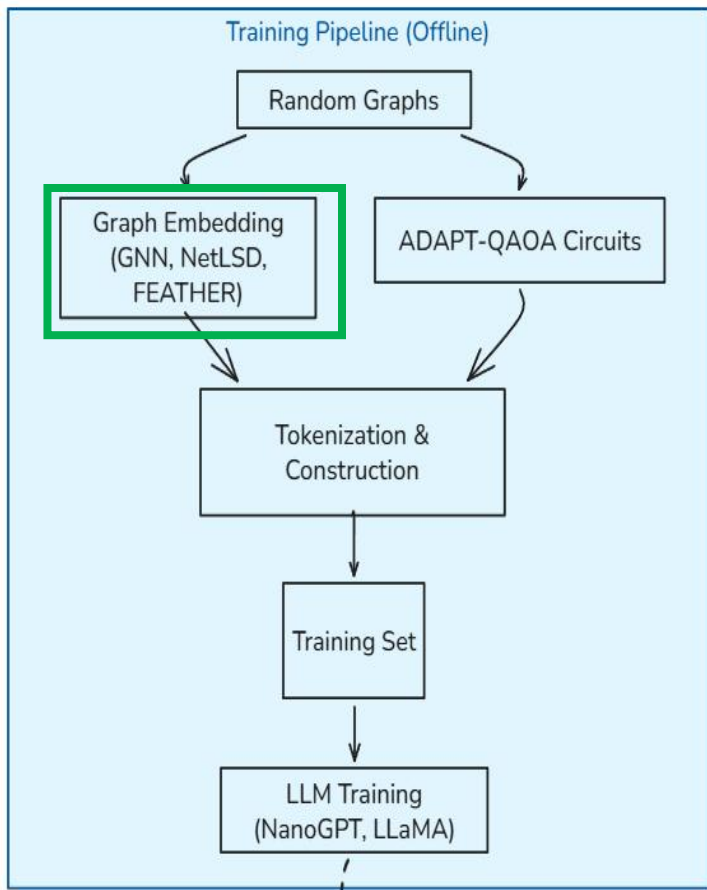
Contrastive Loss

Random Graphs

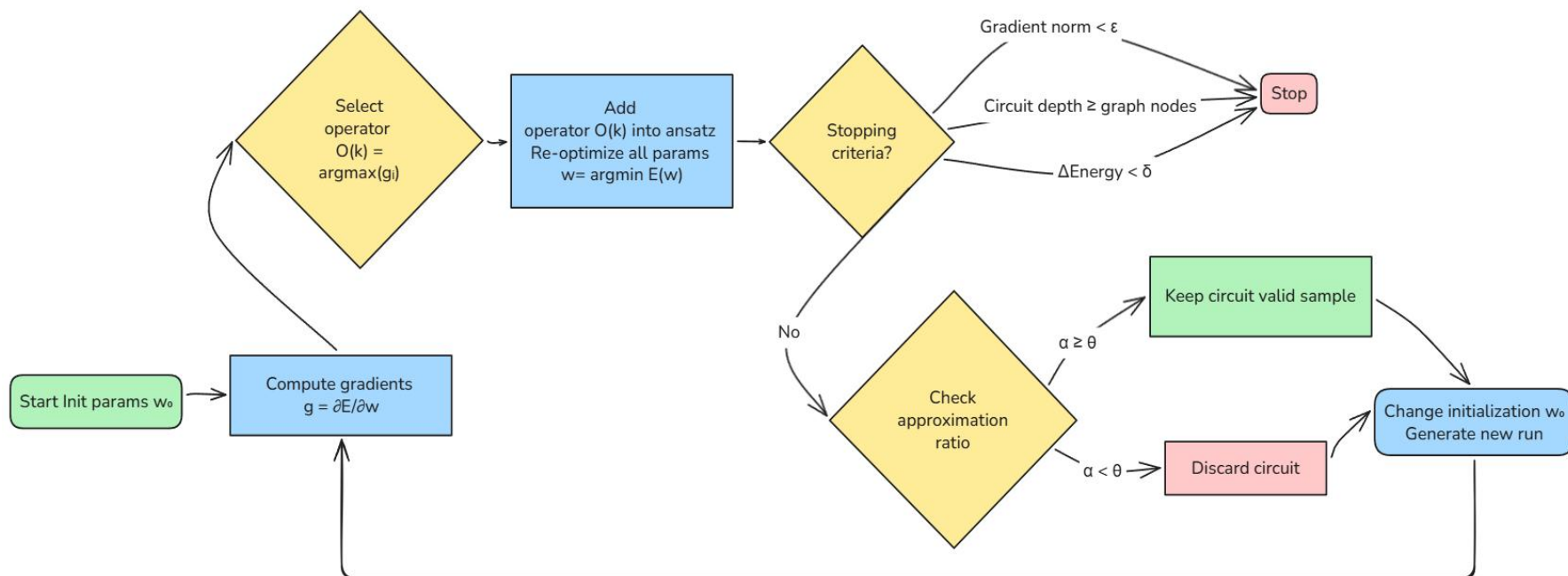
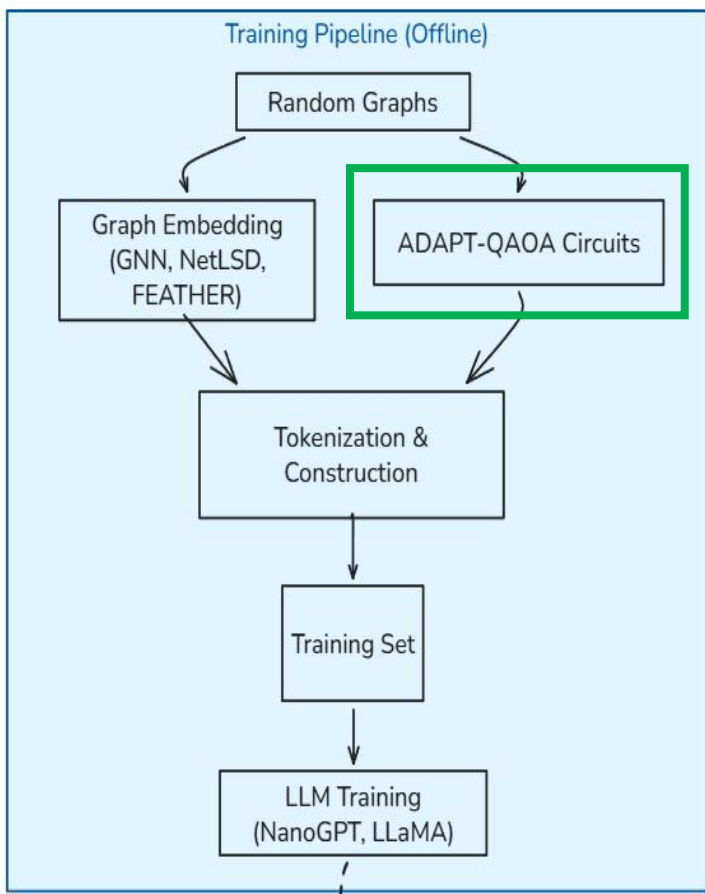


Training Strategy (Self-Supervised)

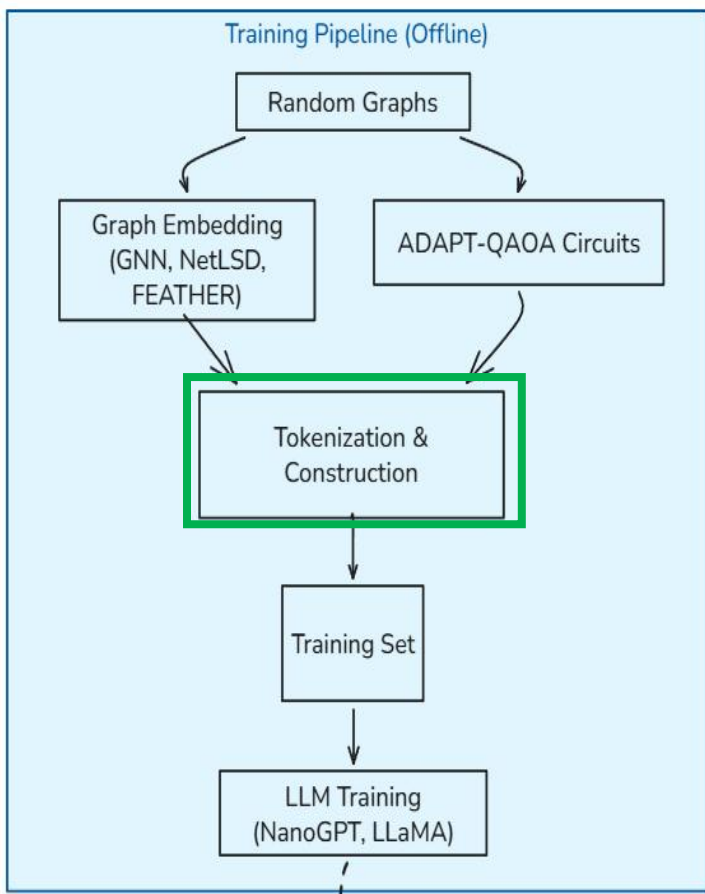
- Data: Synthetic connected Erdős–Rényi graphs
- Random edge weights $\in [0, 1]$, random node features
- Graph augmentations: Edge dropout, Edge weight noise, Node feature noise



ADAPT - QAOA Circuits



Tokenization & Construction



Graph Tokens

Node indices Edge weight

$\langle \text{bos} \rangle, (i_1, j_1), w_{i_1 j_1}, \dots, (i_m, j_m), w_{i_m j_m}, \langle \text{endofgraph} \rangle$

Circuit Tokens

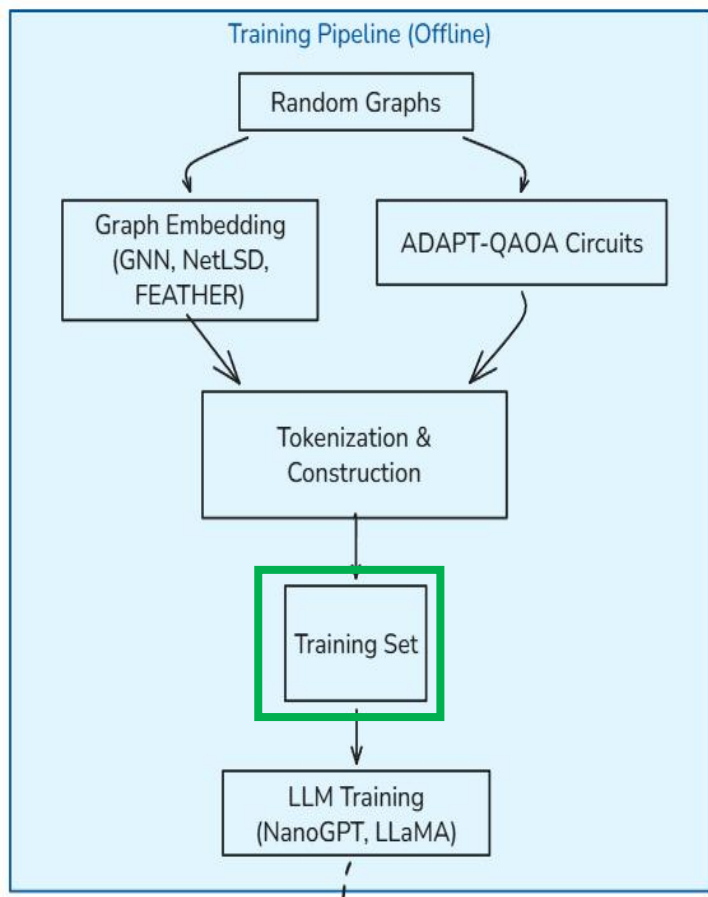
Operator index at layer k

$\langle \text{newlayer} \rangle, o_k, \gamma_k, \beta_k$

Parameters at layer k

Token	Token IDs	Function
<bos>	0	Special token
(0,1)	1	Edge
0.32	2	Edge weight
6.83	4	γ or β
113.0	5	Operator index

Training Set



Final Embedding:

$$X = E_{\text{tok}}(x_{1:T}) + E_{\text{pos}} + e_G$$

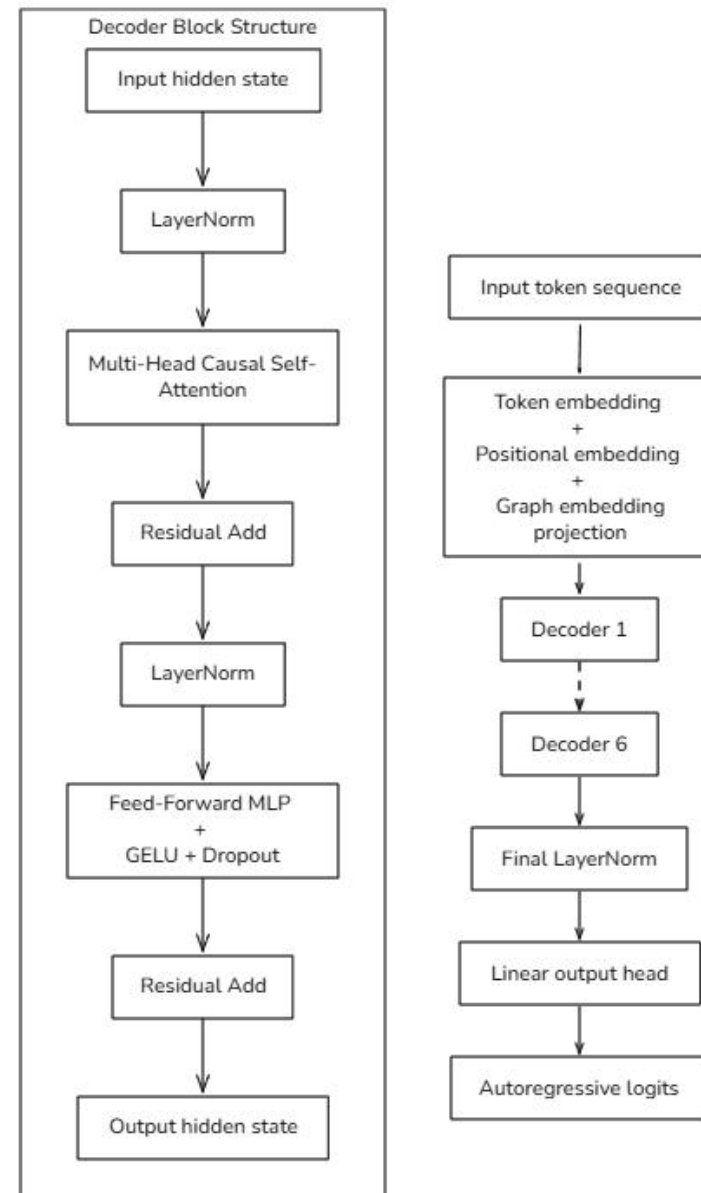
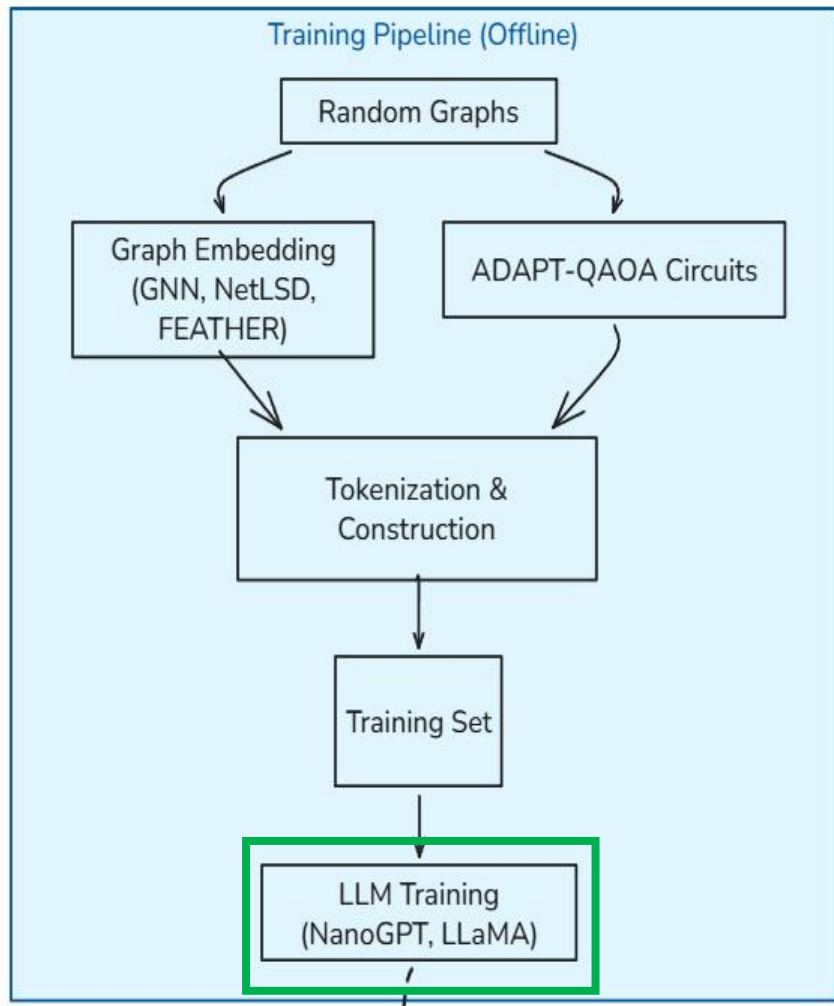
Token Embedding

Positional Embedding

Graph Embedding

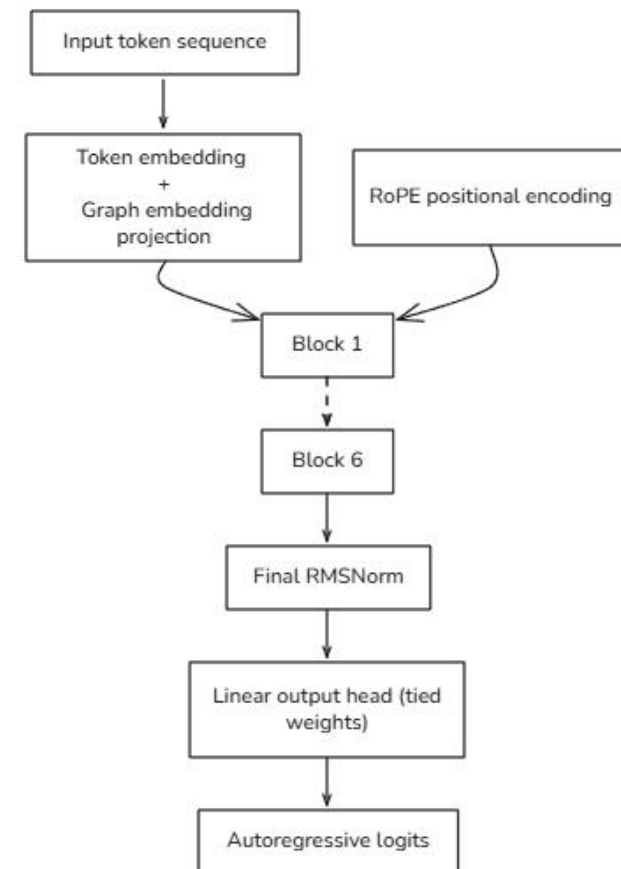
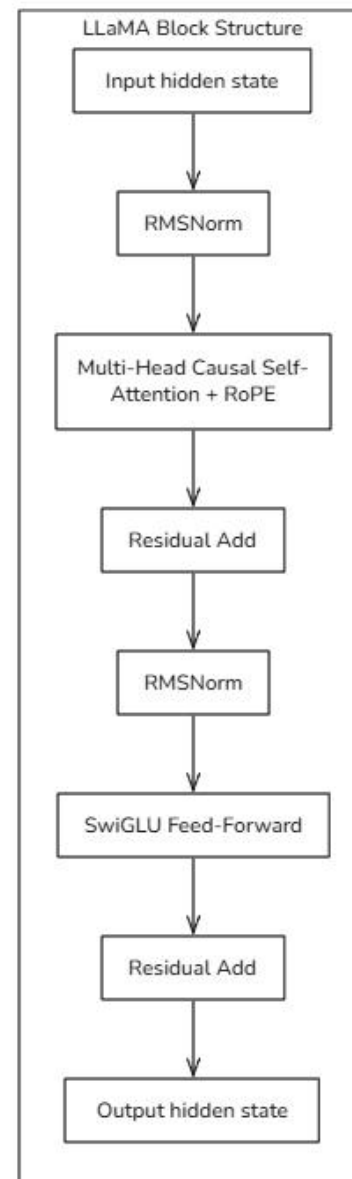
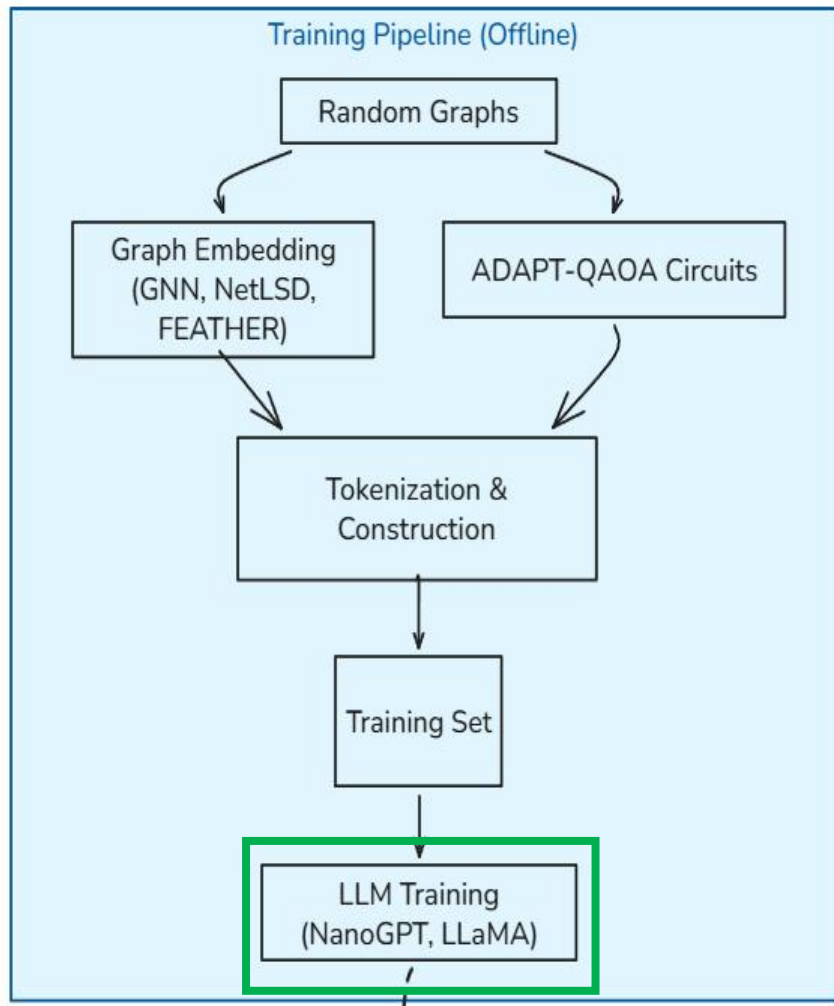
LLM Training

NanoGPT



LLM Training

LLaMA



Experimental Environment

Training Dataset

LLM Models

- There is no publicly available dataset
- Synthetic Erdős–Rényi graphs:
 - 9 nodes: 80,000
 - 10 nodes: 85,887
 - 11 nodes: 50,000
- Train / Val / Test = 8 / 1 / 1

GNN Model

- 500 synthetic graphs
- Nodes: $n \in \{9, 10, 11\}$
- Edge probability: $p \in [0.2, 0.9]$
- 95% weighted | 5% unweighted

Evaluation Dataset

There is no publicly available dataset

Synthetic Erdős–Rényi graphs:

- 100 graphs per size $\rightarrow$ 300 total

Compare the proposed method against baseline approaches:

- Vanilla QAOA
- ADAPT-QAOA

All Dataset generation/ Training/ Testing was done on:

- Laptop: Intel Core i7-10750H CPU, RTX 3060 Laptop GPU (6 GB VRAM)
- Desktop PC: Intel Core i5-13500 CPU, RTX 3060 (12 GB VRAM)

$\rightarrow$ High cost: Data generation takes hours $\rightarrow$ days

Evaluation Metrics

🎯 Approximation Ratio (AR)

- Measures how close the solution is to optimal ($0 \rightarrow 1$, higher is better)
- Evaluated on held-out graphs during training

📉 Validation Loss

- Cross-entropy on validation set
- Used to detect overfitting & select best checkpoint

⚠️ Validity — Error Rate

- % of invalid generated circuits
- Invalid if: parsing fails / wrong structure / energy eval fails

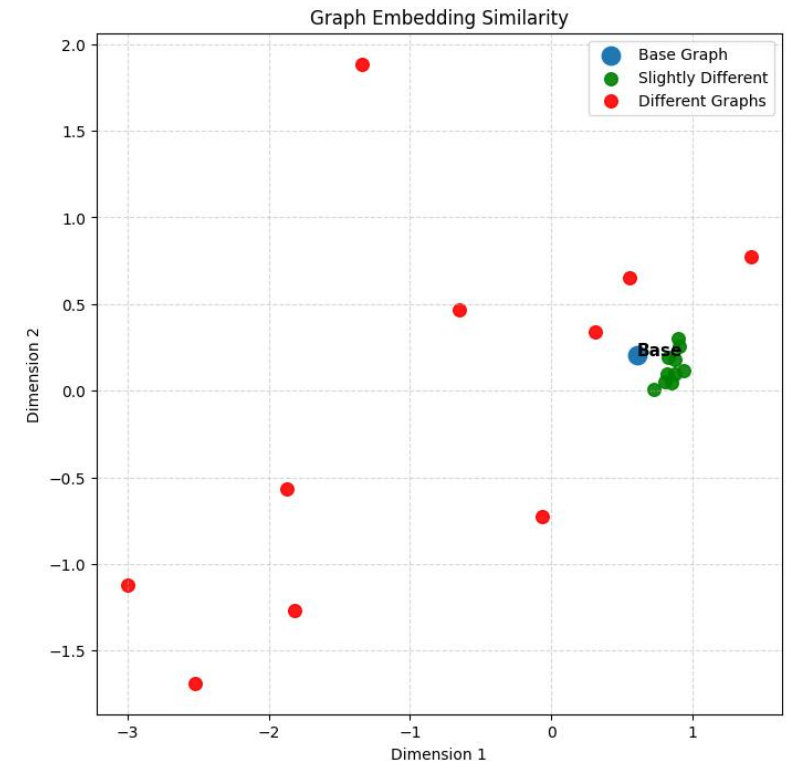
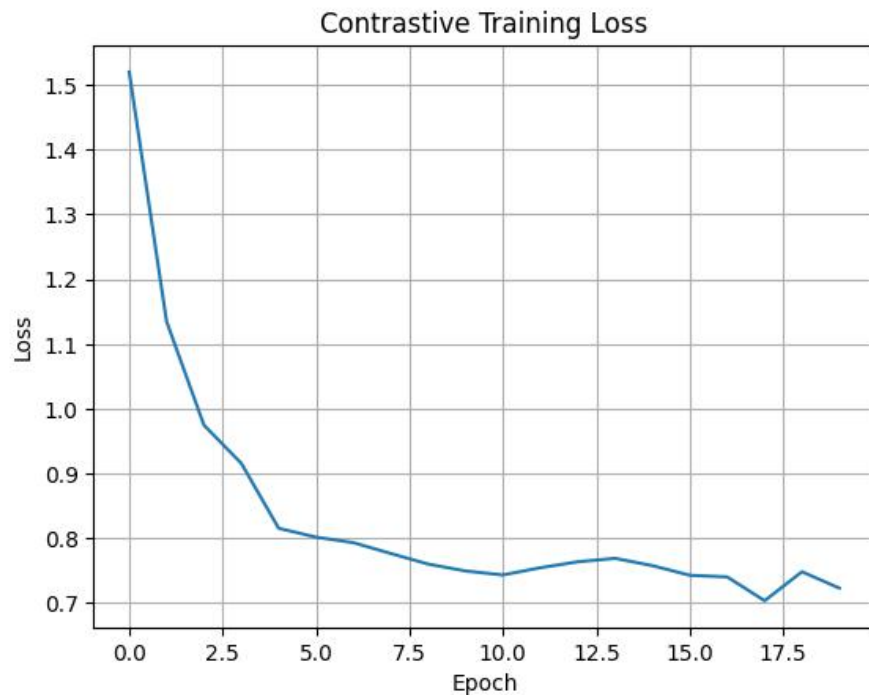
→ Checks if model learns correct syntax & structure

⚡ Circuit Layers

- Number of layers = number of parameter pairs (γ, β)
- Trade-off:
 - Fewer → ⚡ faster, cheaper
 - More → 🎯 better expressiveness
- Goal: high performance with fewer layers

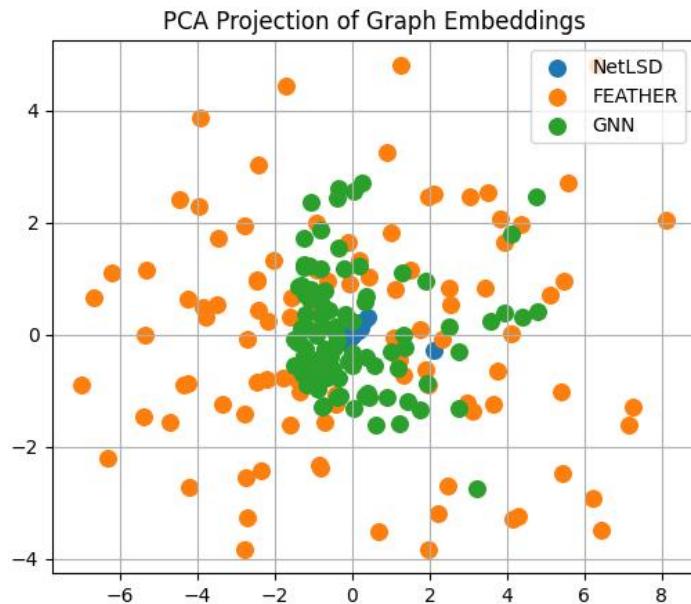
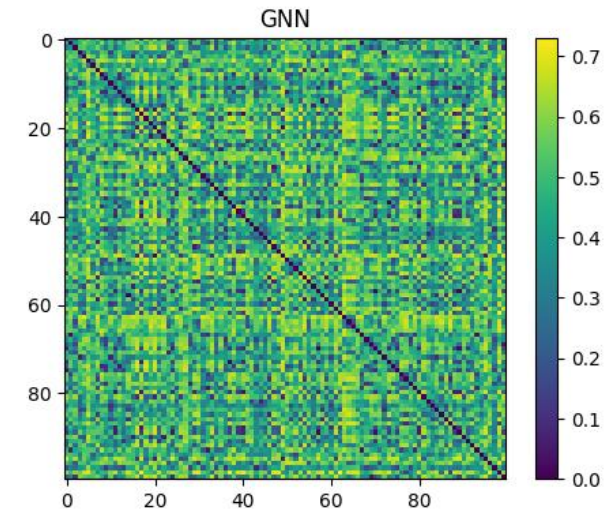
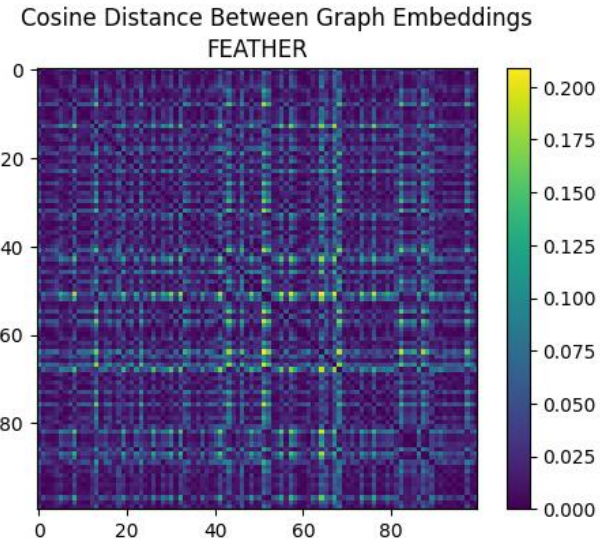
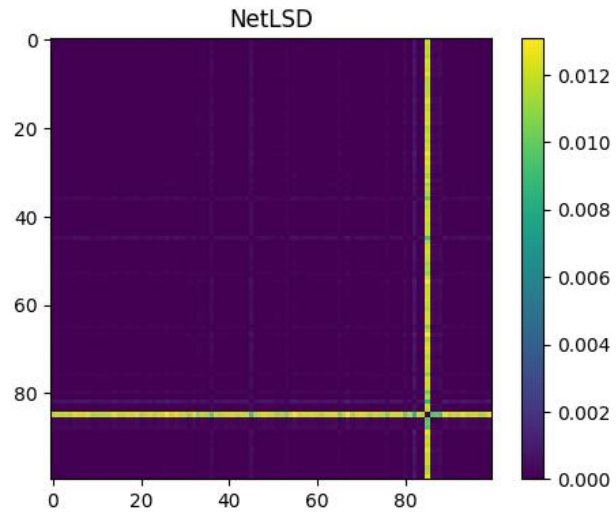
Result & Analysis

GNN Training Results

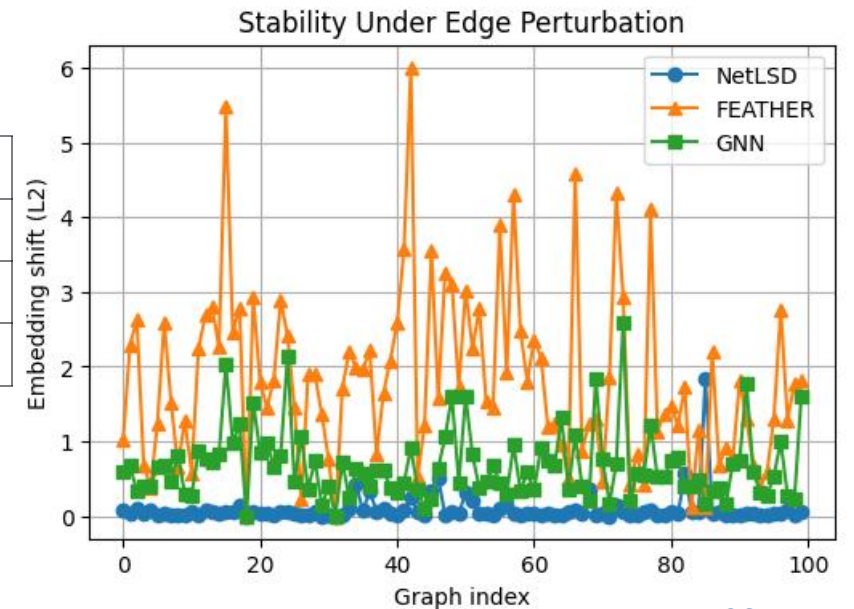


- No signs of divergence or exploding gradients are observed throughout training
- Fast & stable convergence: Loss drops quickly ($\approx 1.5 \rightarrow < 0.8$ in 5 epochs) and stabilizes ($\sim 0.65\text{--}0.75$)
→ efficient learning with no instability
- Robust embedding space where geometric proximity correlates directly with structural similarity

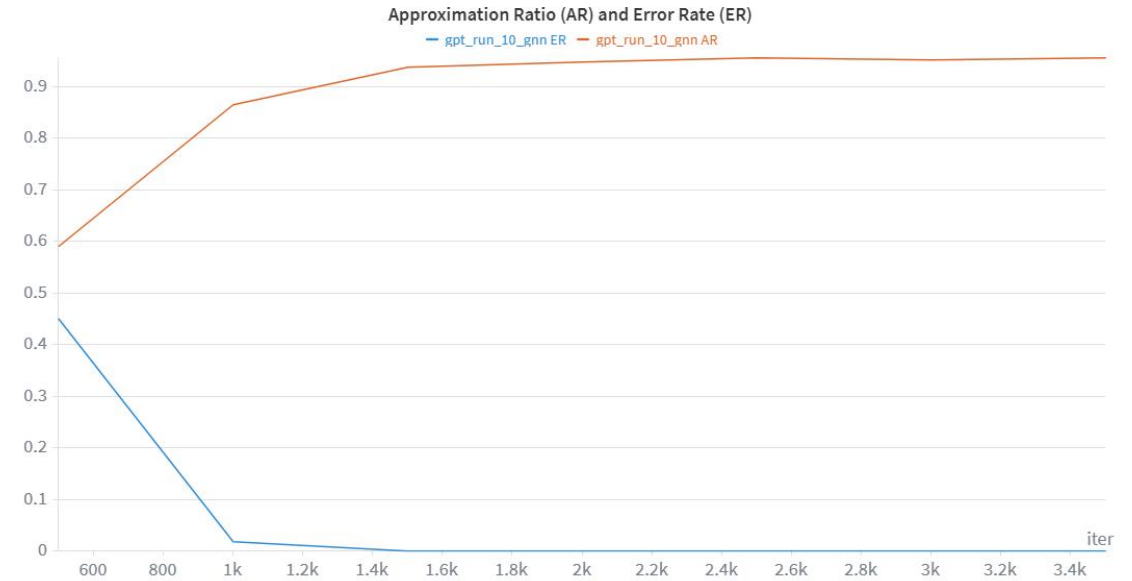
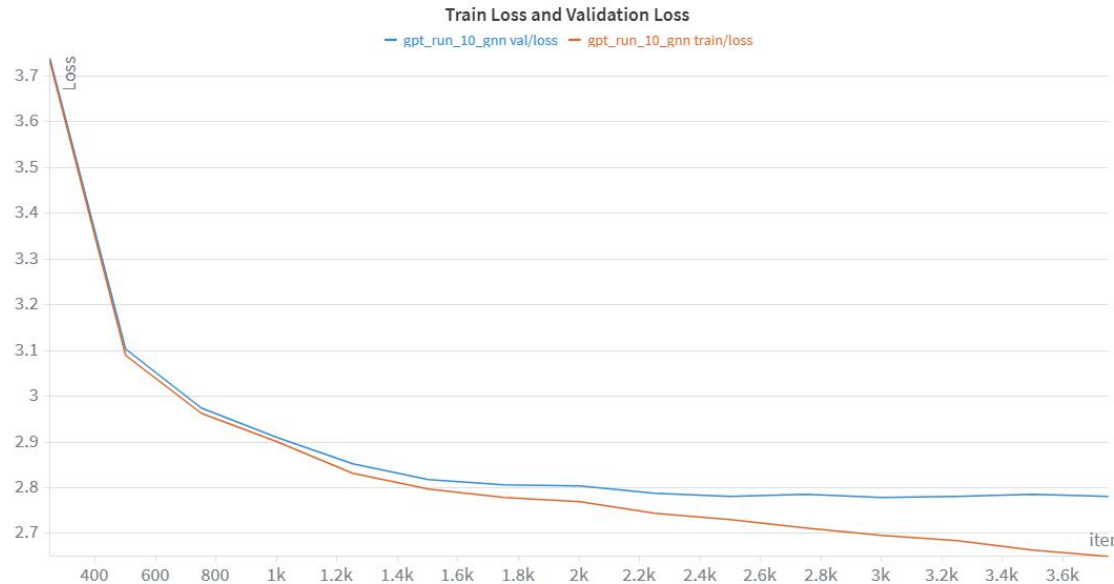
Embedding Analysis



Method	Mean Per-Dimension Std
NetLSD	0.009303
GNN	0.107253
FEATHER	0.168366



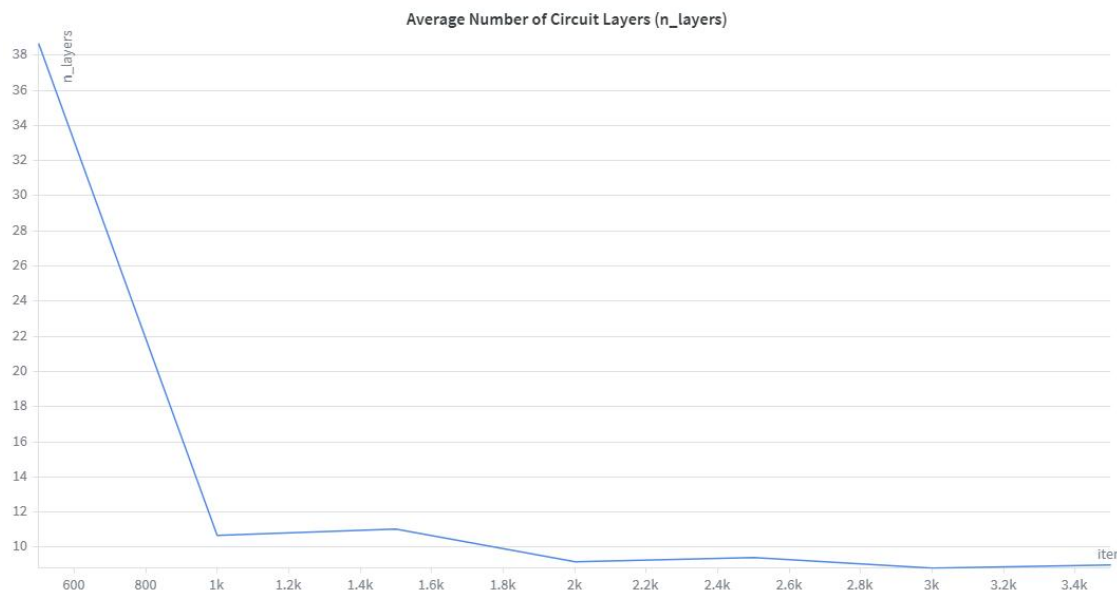
Training and Validation Loss & Learning Dynamics



The close alignment between the training and validation curves indicates stable optimization and suggests that overfitting is limited in this configuration

- Rapid convergence in early stages
- Stabilization of performance
- Early acquisition of high-quality patterns:

Training and Validation Loss & Learning Dynamics

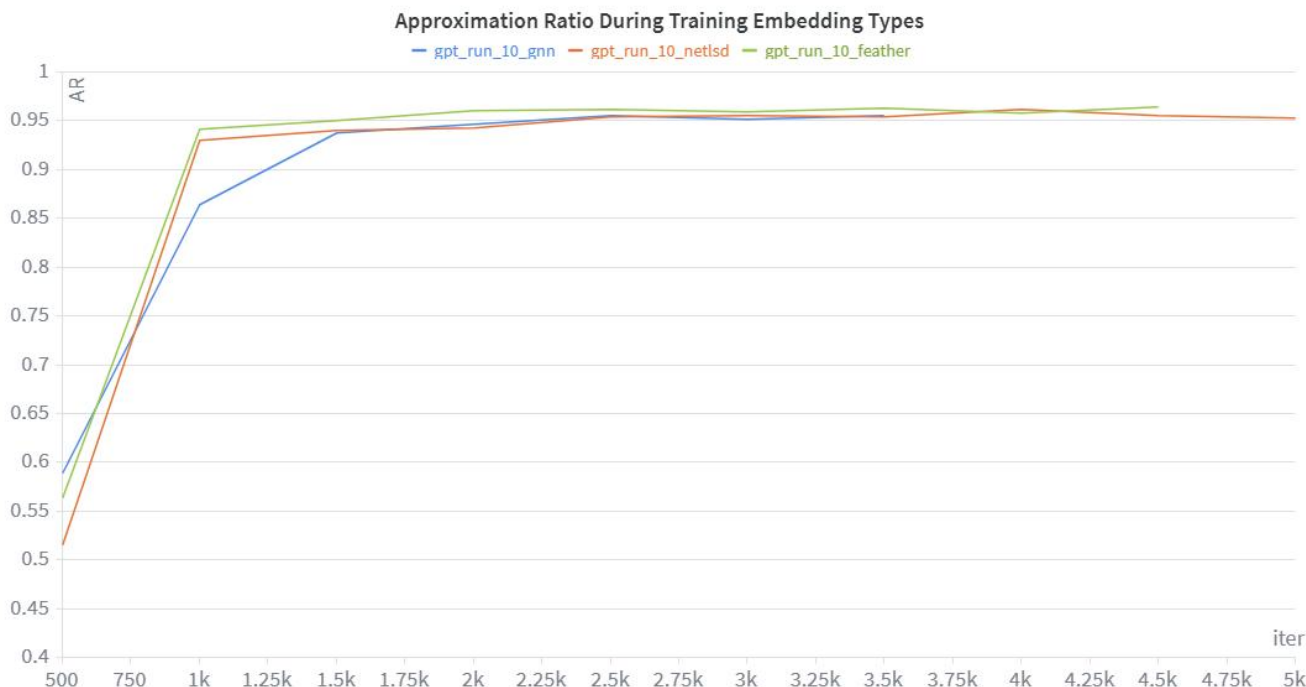


- Model learns to generate more compact circuits while maintaining high performance.
- The reduction in circuit depth reflects improved efficiency



- NanoGPT achieves faster convergence and earlier stabilization
- LLaMA provides smoother but slower performance improvement

Training and Validation Loss & Learning Dynamics

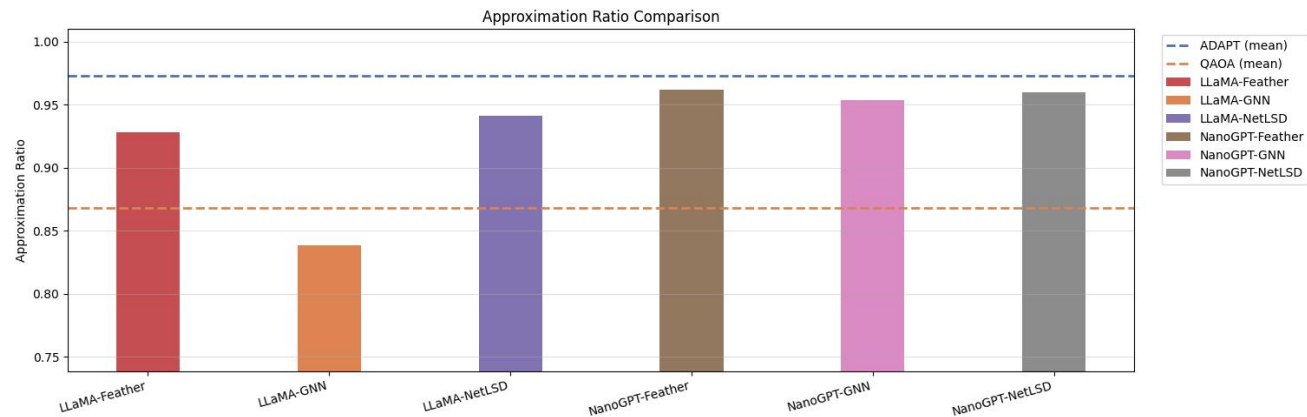


- **Fast early learning (all methods):** AR improves rapidly in first ~1,000 iterations → all embeddings capture useful structure
 - **GNN performs best:** Highest AR, faster convergence, and most stable → more expressive & task-relevant
 - **NetLSD & FEATHER slightly weaker:** High but lower plateau + less stable
- fixed descriptors < learned embeddings

Approximation Ratio

Nodes	Adapt mean	Qaoa mean	LLaMA-Feather	LLaMA-GNN	LLaMA-NetLSD	NanoGPT-Feather	NanoGPT-GNN	NanoGPT-NetLSD
9	0.973	0.861	0.941	0.927	0.903	0.955	0.951	<u>0.957</u>
10	0.973	0.868	0.928	0.839	0.941	<u>0.962</u>	0.954	0.96
11	0.971	0.874	0.913	0.924	0.93	0.961	0.95	<u>0.968</u>

Approximation Ratio (AR) for Vanilla Qaoa, Adapt-Qaoa and This work's methods. Results Are Presented as Mean Values Across 100 Graphs For Each n nodes graph. The best and second-best values in each row are highlighted in **bold** and underlined, respectively. Methods proposed in this work are additionally indicated with a **green** background



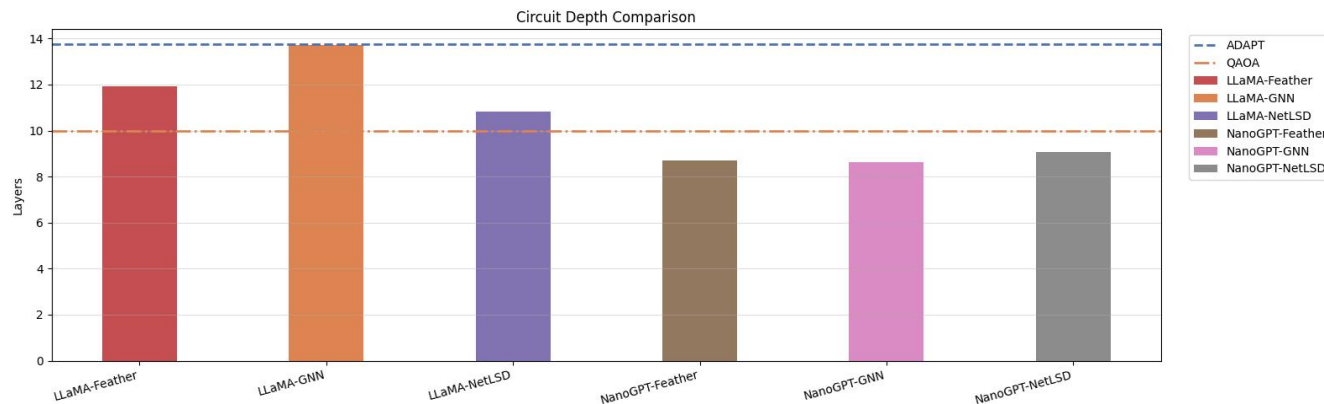
Approximation Ratio comparison for 10-node graphs

- **Strong performance vs baselines:** Significantly outperforms Vanilla QAOA and achieves near ADAPT-QAOA results across all sizes
- **NanoGPT models lead:** Most stable and high-performing ($AR \approx 0.95-0.968$)
→ good scalability with graph size
- **LLaMA less consistent:** Higher variability, occasional drops → less reliable across embeddings and sizes

Circuit Depth Analysis

Number of Circuit Layers								
Nodes	Adapt mean	Qaoa mean	LLaMA-Feather	LLaMA-GNN	LLaMA-NetLSD	NanoGPT-Feather	NanoGPT-GNN	NanoGPT-NetLSD
9	13.138	9	<u>8.806</u>	9.858	9.598	7.952	9.264	8.85
10	13.738	10	11.92	13.716	10.832	<u>8.688</u>	8.638	9.072
11	15.407	11	14.896	15.444	12.412	<u>8.908</u>	8.244	11.85

Number of Circuit Layers for Vanilla Qaoa, Adapt-Qaoa and This work's methods. Results Are Presented as Mean Values Across 100 Graphs For Each n nodes graph. The best and second-best values in each row are highlighted in **bold** and underlined, respectively. Methods proposed in this work are additionally indicated with a **green** background.



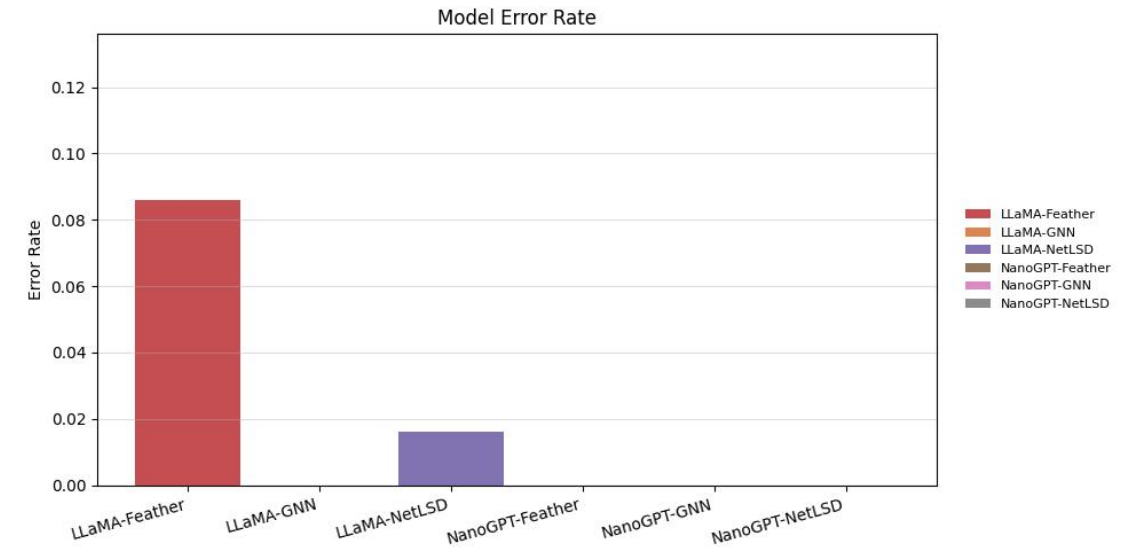
Circuit depth comparison for 10-node graphs

- **Better quality–depth trade-off:** Achieves high performance with much shallower circuits than ADAPT-QAOA
- **NanoGPT = most efficient:** Produces smallest circuit depth (~8–9 layers) → even lower than Vanilla QAOA
- **LLaMA less efficient:** Requires deeper, more variable circuits, sometimes exceeding ADAPT-QAOA depth

Error Rate Analysis

Error Rate						
Nodes	LLaMA-Feather	LLaMA-GNN	LLaMA-NetLSD	NanoGPT-Feather	NanoGPT-GNN	NanoGPT-NetLSD
9	0	0	0	0	0	0
10	0.086	0	0.016	0	0	0
11	0	0	0	0	0	0

Error Rate for This work's methods. Results Are Presented as Mean Values Across 100 Graphs For Each n nodes graph



Error Rate comparison for 10-node graphs

- **Near-zero error overall:** Most models achieve 0 error, showing high reliability across graph sizes
- **NanoGPT = most stable:** Consistently zero error across all sizes (9–11 nodes):
- **LLaMA more variable:** Shows error spikes (e.g., n=10) → less consistent, though recovers at larger size

Embedding Types Analysis

Model Type	Model Name	Embedding Method	Adapt AR Mean	Adapt Layers	Model AR	Model Error Rate	Model Layers
LLaMA	LLaMA-Feather	Feather	0.973	13.738	0.928	0.086	11.920
	LLaMA-GNN	GNN	0.973	13.738	0.838	<u>0.002</u>	13.988
	LLaMA-NetLSD	NetLSD	0.973	13.738	0.941	0.016	10.832
Nano-GPT	NanoGPT-Feather	Feather	0.973	13.738	0.962	0.000	<u>8.688</u>
	NanoGPT-GNN	GNN	0.973	13.738	0.954	0.000	8.638
	NanoGPT-NetLSD	NetLSD	0.973	13.738	<u>0.960</u>	0.000	9.072

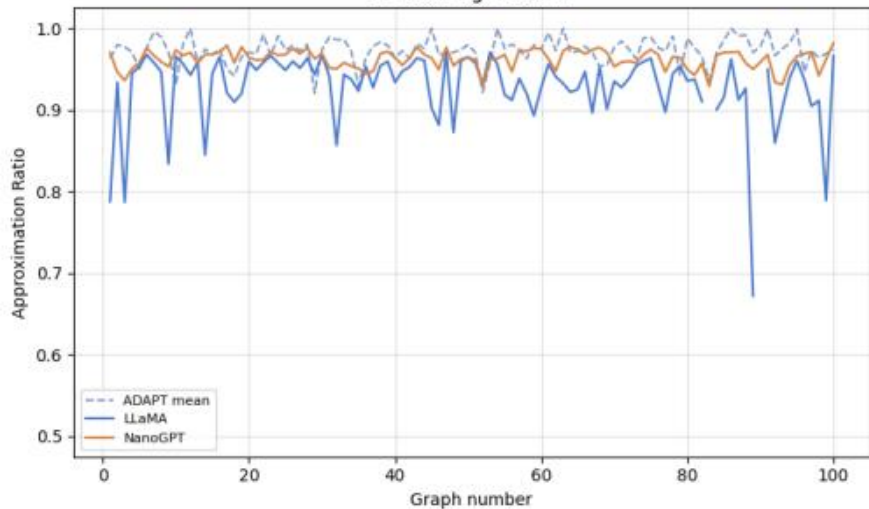
- **NetLSD & FEATHER strongest overall:** High and stable AR (≈ 0.96 in NanoGPT); FEATHER less reliable in LLaMA
- **GNN depends on model:** Strong in NanoGPT (high AR, zero error) but drops significantly in LLaMA → model–embedding mismatch

Performance Comparison of Embedding Methods Across LLaMA and NanoGPT Architectures. Results Are Presented as Mean Values across 100 Graphs for 10-node graph. The best and second-best values in each column are highlighted in **bold** and underlined, respectively

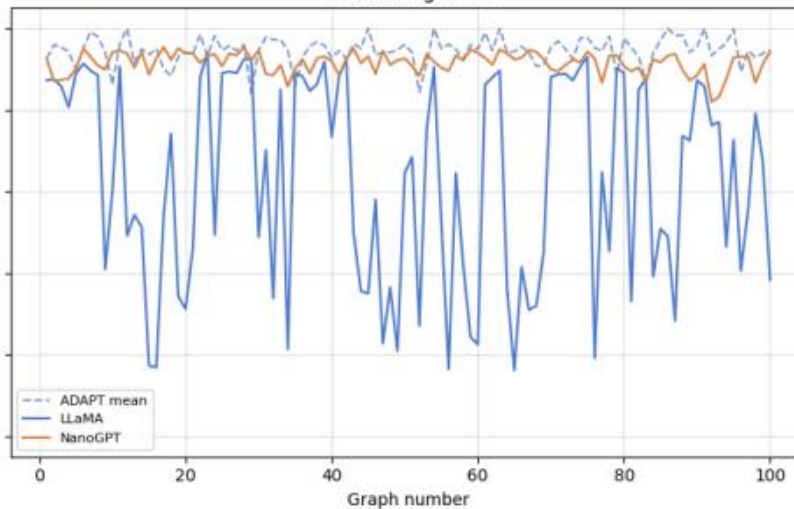
Embedding Types Analysis

Per-graph AR by Embedding Method

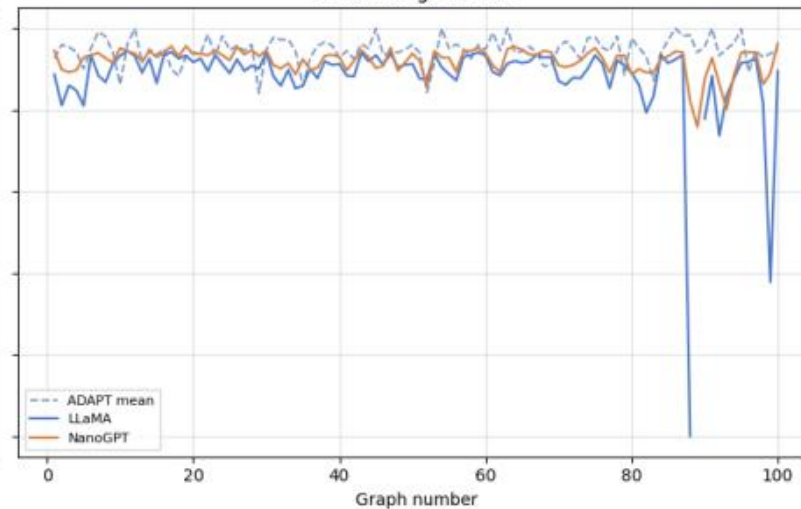
Embedding: Feather



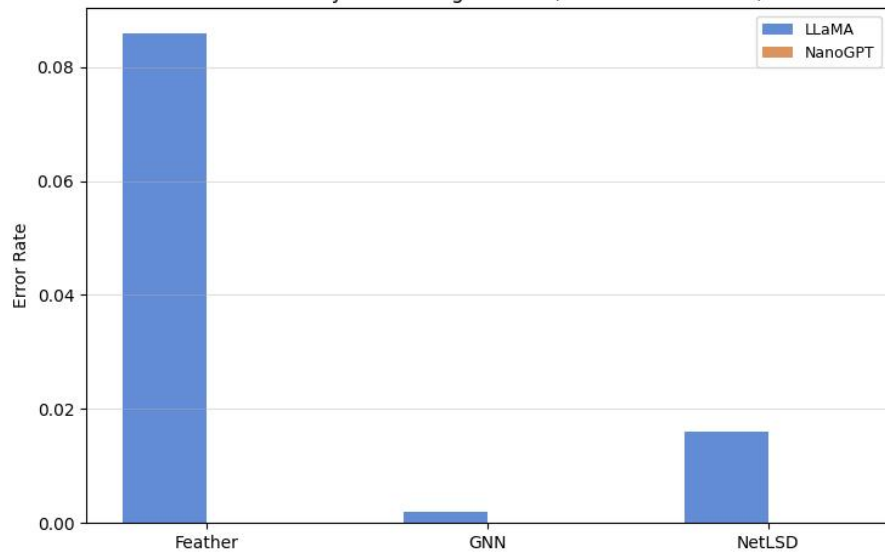
Embedding: GNN



Embedding: NetLSD



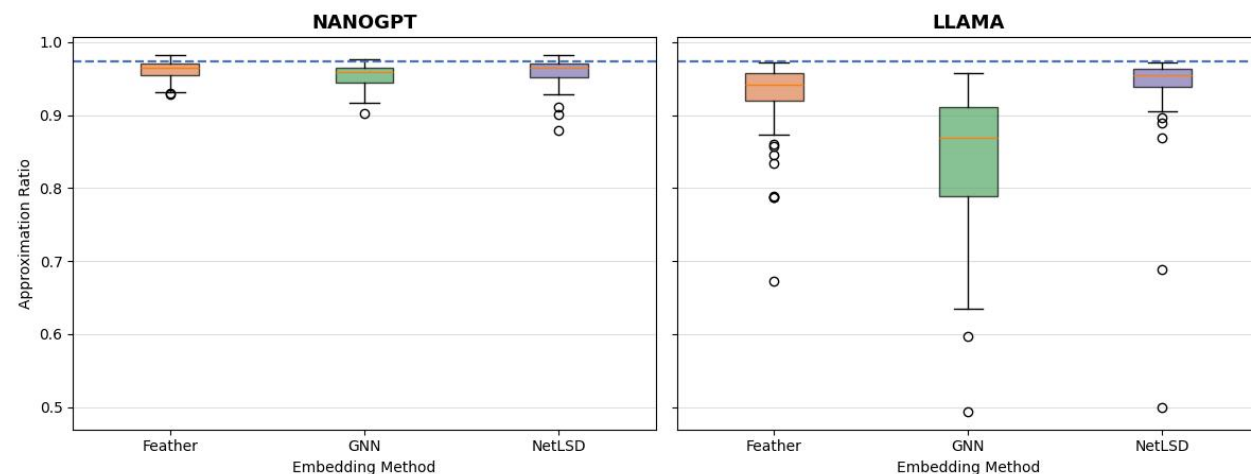
Error Rate by Embedding Method (bars = architecture)



Embedding Types Analysis

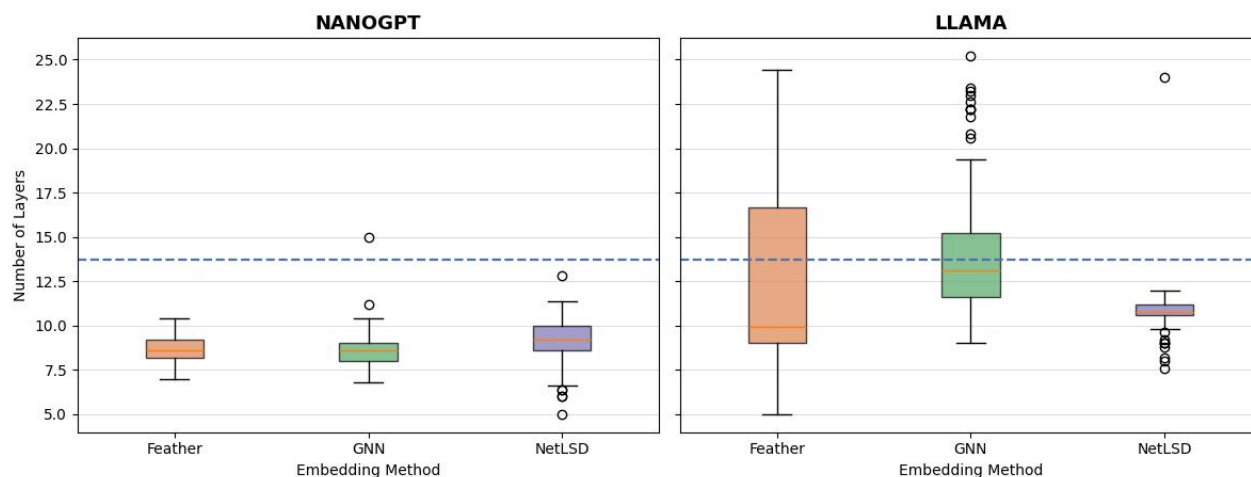
AR Distribution by Embedding Method

Feather embedding GNN embedding NetLSD embedding ADAPT average AR



Layer Distribution by Embedding Method

Feather embedding GNN embedding NetLSD embedding ADAPT average layers



- NetLSD provides the most stable and consistent performance across different graph instances.
 - GNN captures richer structural information but introduces higher variability and less stable optimization behavior.
 - FEATHER shows larger performance fluctuations and produces the highest error rate in the LLaMA architecture.
- > Model architecture has a much larger impact than embedding choice.

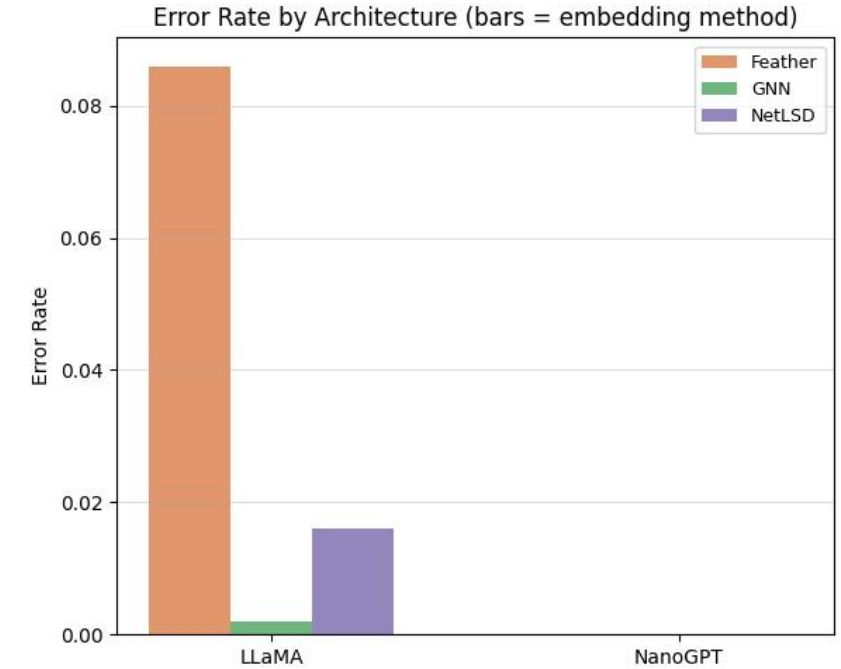
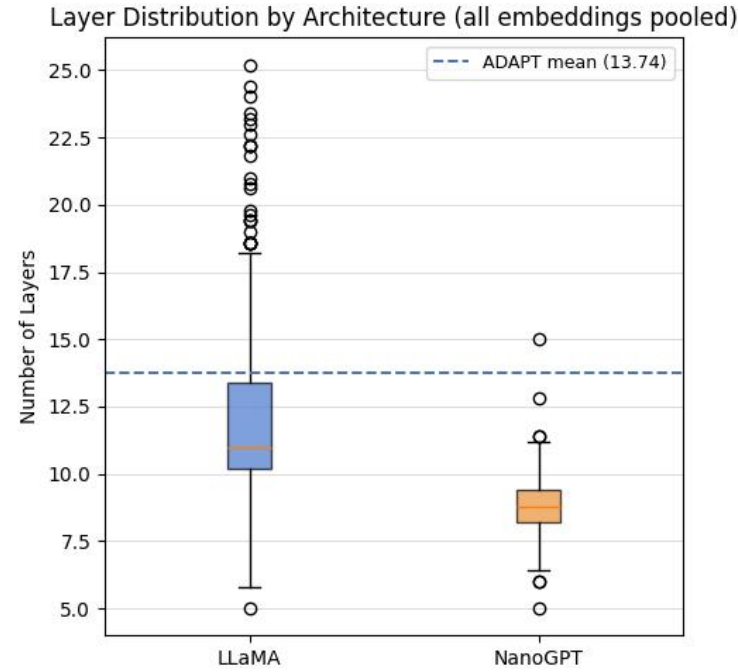
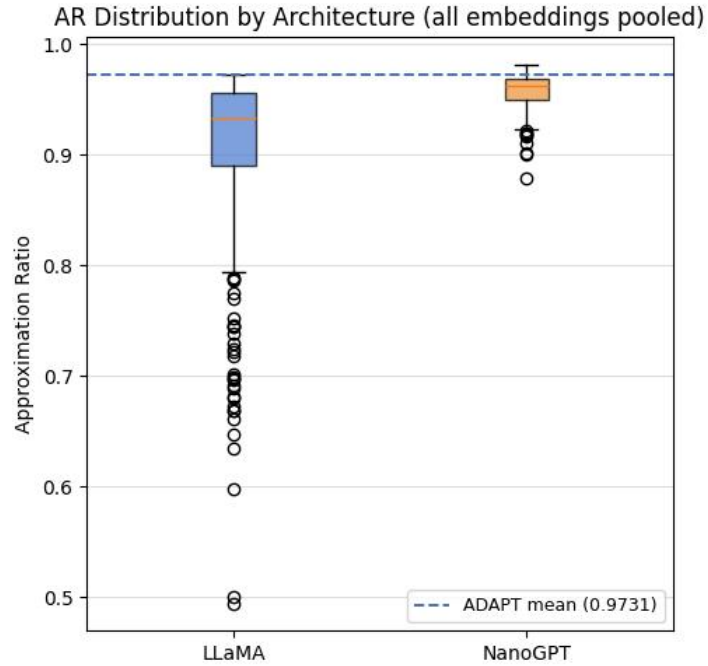
Model Architecture Analysis

Model Type	Model Name	Embedding Method	Adapt AR Mean	Adapt Layers	Model AR	Model Error Rate	Model Layers
LLaMA	LLaMA-Feather	Feather	0.973	13.738	0.928	0.086	11.920
	LLaMA-GNN	GNN	0.973	13.738	0.838	<u>0.002</u>	13.988
	LLaMA-NetLSD	NetLSD	0.973	13.738	0.941	0.016	10.832
Nano-GPT	NanoGPT-Feather	Feather	0.973	13.738	0.962	0.000	<u>8.688</u>
	NanoGPT-GNN	GNN	0.973	13.738	0.954	0.000	8.638
	NanoGPT-NetLSD	NetLSD	0.973	13.738	<u>0.960</u>	0.000	9.072

- **NanoGPT outperforms LLaMA:** Higher AR (≈ 0.954 – 0.962) + zero error across all embeddings
- **More stable & robust:** NanoGPT consistent across settings $\rightarrow$ less sensitive to embeddings/graphs
- **LLaMA more variable:** Lower AR (≈ 0.838 – 0.941) + non-zero errors $\rightarrow$ less reliable overall

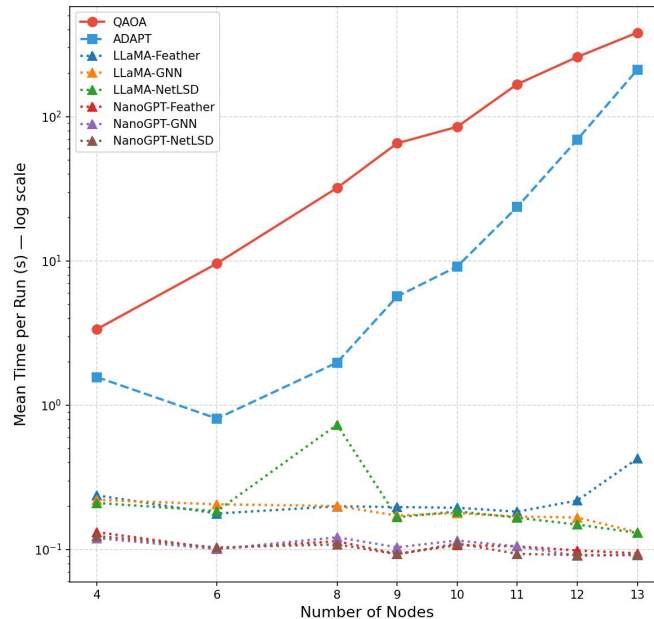
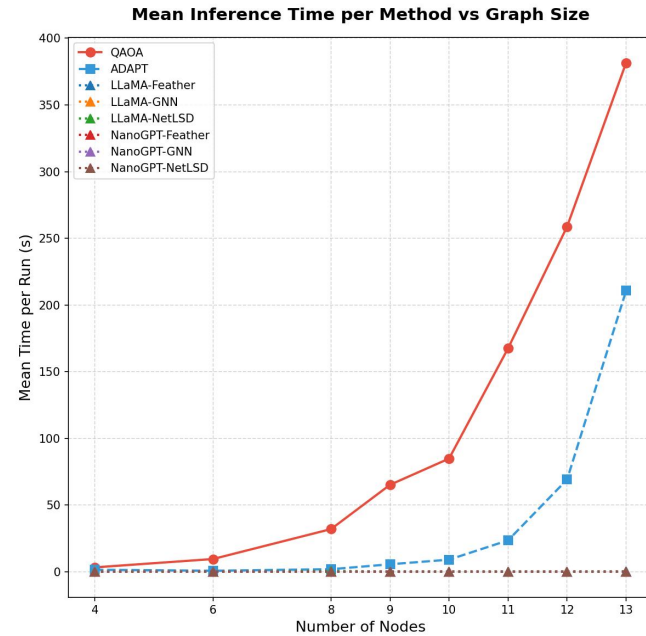
Performance Comparison of Embedding Methods Across LLaMA and NanoGPT Architectures. Results Are Presented as Mean Values across 100 Graphs for 10-node graph. The best and second-best values in each column are highlighted in **bold** and underlined, respectively

Model Architecture Analysis



- **Architecture simplicity → better stability:** NanoGPT's standard Transformer (LayerNorm + GELU + learned positions) fits structured sequence tasks → smoother optimization & stronger generalization
- **LLaMA design introduces sensitivity:** RoPE + RMSNorm + SwiGLU add complexity → less aligned with discrete circuit tokens → more unstable & sensitive to inputs
- **Better embedding integration in NanoGPT:** Uses graph embeddings more consistently → robust across embeddings, while LLaMA amplifies input variation → performance drops

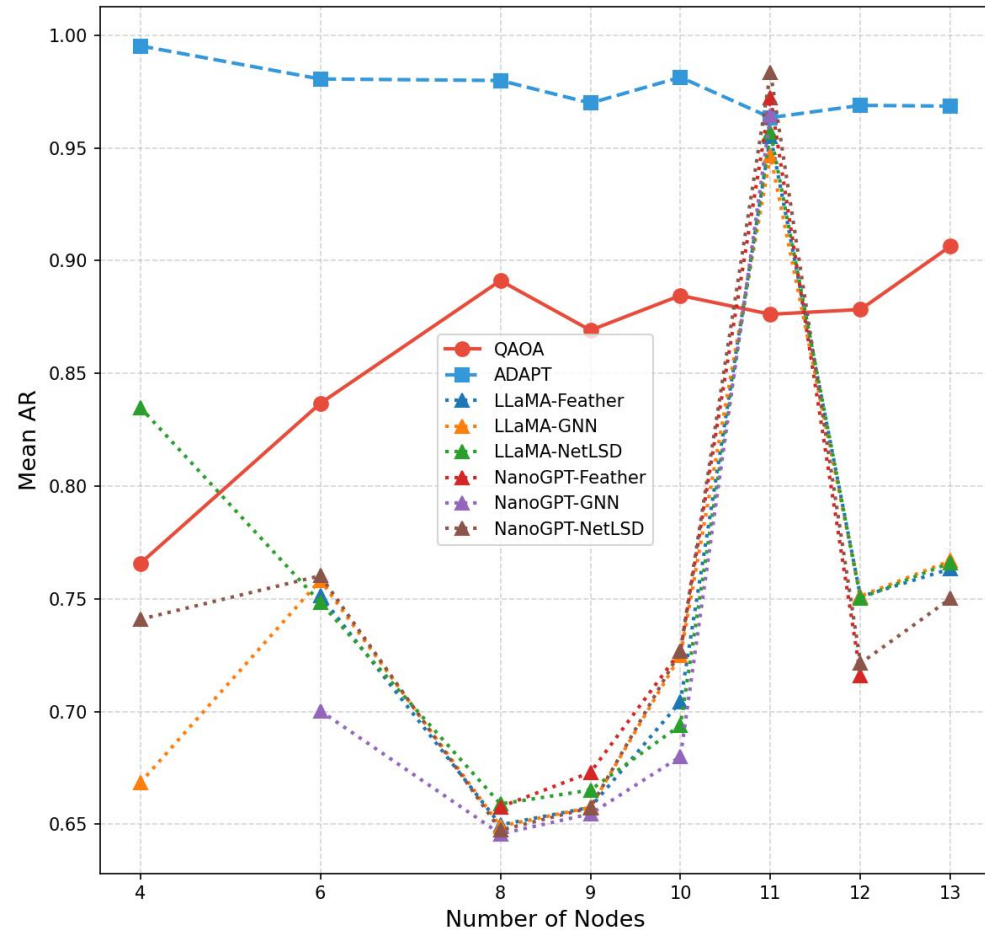
Inference Time Analysis Across Graph Sizes



- QAOA and ADAPT-QAOA scale poorly with graph size, with runtime increasing rapidly (up to ~167s), while the proposed LLM-GNN methods remain nearly constant (~0.08–0.25s).
- Quantum methods require iterative optimization and circuit evaluations ($\approx$ exponential in 2^n), whereas the LLM-GNN framework uses a single forward pass, shifting cost to offline training.
- NanoGPT variants are consistently faster than LLaMA due to lighter architecture, and different graph embeddings (Feather, GNN, NetLSD) have minimal impact on inference time.

Approximation Ratio Performance Across Graph Sizes

Mean Approximation Ratio per Method vs Graph Size

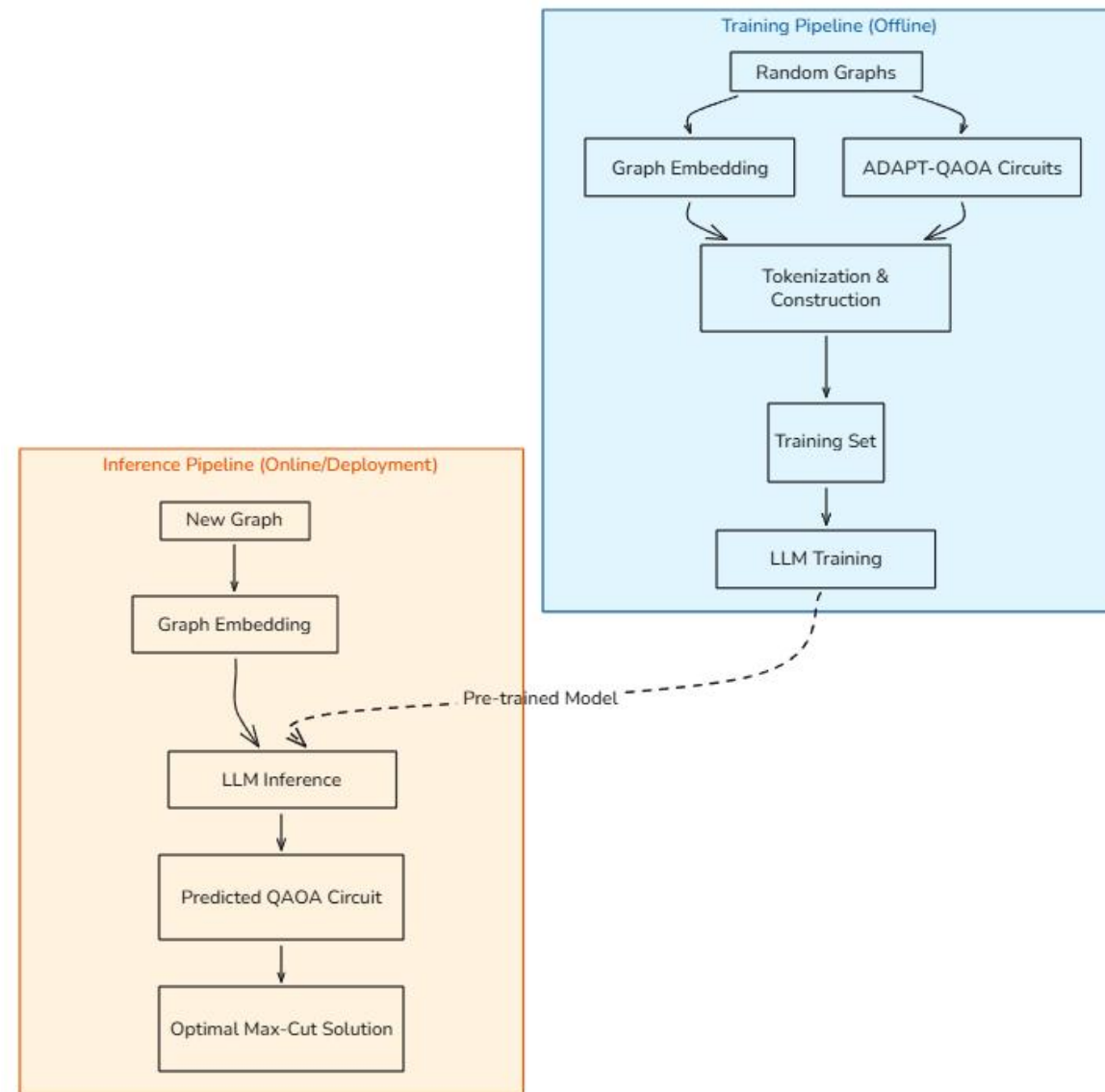


- ADAPT-QAOA and vanilla QAOA remains stable and improves slightly with size, confirming strong and reliable solution quality despite higher computational cost.
- The LLM-GNN framework performs best at 11 nodes (AR ≈ 0.95 – 0.98), the training size, but drops significantly (≈ 0.65 – 0.76) on other sizes, showing limited generalization beyond the training distribution.
- NanoGPT vs LLaMA trade-off: NanoGPT achieves the highest peak performance but fails to generate valid circuits for some sizes, indicating poor robustness. LLaMA is more consistent across sizes, suggesting better generalization and higher potential if trained on more diverse, multi-size datasets.

Conclusion & Future Work

Contributions

- **Fast Learning Convergence:** Generative-predictive framework (NanoGPT, LLaMA) achieves $AR > 0.9$ early, while $ER \rightarrow 0$ with stable training
- **Joint Optimization:** Simultaneously improves solution quality and circuit structure, reducing QAOA layers while maintaining performance
- **Graph-Aware Generation:** Integrates embeddings (NetLSD, FEATHER, GNN) to guide circuit design & parameter initialization
- **Structural Convergence:** Circuit depth stabilizes $\rightarrow$ model learns efficient, compact configurations
- **Robust Performance:** Extensive experiments show stable AR, low ER, and optimized circuit depth across datasets



Conclusions

- GNN embeddings: best balance (vs NetLSD stable / FEATHER sensitive)
- NanoGPT > LLaMA: faster convergence, lower error, more stable
- Achieves near ADAPT-QAOA performance
→ with much lower circuit depth
- Combines learning + quantum optimization effectively Improves over Vanilla QAOA while reducing complexity vs ADAPT
- Promising scalable approach for graph-based quantum problems

Future Work

End-to-End Learning

- Jointly train GNN + LLM (instead of separate stages)
- Align embeddings with circuit generation objective
- Improve representation quality + final performance

Generalization Across Sizes

- Current: one model per graph size
- Future: single scalable model
- Use size-invariant encoding / curriculum learning

Task-Aware Objectives

- Move beyond token prediction loss
- Incorporate Approximation Ratio (AR), Error Rate (ER)
- Use multi-objective / RL-based optimization

Scalability & Real-World Deployment

- Extend to larger graphs & other problems
- Move toward practical quantum applications
- Consider hardware constraints (noise, connectivity)

BIBLIOGRAPHY

- [1] L. Huynh, J. Hong, A. Mian, H. Suzuki, Y. Wu, and S. Camtepe, “Quantum-Inspired Machine Learning: a Survey,” Aug. 2023, [Online]. Available: <http://arxiv.org/abs/2308.11269>
- [2] K. Zaman, A. Marchisio, M. A. Hanif, and M. Shafique, “A Survey on Quantum Machine Learning: Current Trends, Challenges, Opportunities, and the Road Ahead,” Oct. 2023, [Online]. Available: <http://arxiv.org/abs/2310.10315>
- [3] Y. Liao and C. Ferrie, “GPT on a Quantum Computer,” Mar. 2024, [Online]. Available: <http://arxiv.org/abs/2403.09418>
- [4] Tyagin, I., Farag, M. H., Sherbert, K., Shirali, K., Alexeev, Y., & Safro, I. (2025). “QAOA-GPT: Efficient Generation of Adaptive and Regular Quantum Approximate Optimization Algorithm Circuits.” ArXiv. <https://arxiv.org/abs/2504.16350>
- [5] Jiang, L., Zhang, C., & Chen, F. (2025). “QSeer: A Quantum-Inspired Graph Neural Network for Parameter Initialization in Quantum Approximate Optimization Algorithm Circuits.” ArXiv. <https://arxiv.org/abs/2505.06810>
- [6] Zhou, J., Cui, G., Hu, S., Zhang, Z., Yang, C., Liu, Z., Wang, L., Li, C., & Sun, M. (2018). “Graph Neural Networks: A Review of Methods and Applications.” ArXiv. <https://arxiv.org/abs/1812.08434>
- [7] Zhu, L., Tang, H. L., Barron, G. S., A., F., Mayhall, N. J., Barnes, E., & Economou, S. E. (2020). “An adaptive quantum approximate optimization algorithm for solving combinatorial problems on a quantum computer.” ArXiv. <https://arxiv.org/abs/2005.10258>
- [8] Wang, Z., Hadfield, S., Jiang, Z., & Rieffel, E. G. (2017). “Quantum Approximate Optimization Algorithm for MaxCut: A Fermionic View.” Physical Review A, 97(2), 022304. <https://doi.org/10.1103/PhysRevA.97.022304>
- [9] Liang, Z., Liu, G., Liu, Z., Cheng, J., Hao, T., Liu, K., Ren, H., Song, Z., Liu, J., Ye, F., & Shi, Y. (2024). Graph learning for parameter prediction of quantum approximate optimization algorithm. arXiv. <https://doi.org/10.48550/arXiv.2403.03310>
- [10] Rozemberczki, B., & Sarkar, R. (2020). “Characteristic Functions on Graphs: Birds of a Feather, from Statistical Descriptors to Parametric Models.” ArXiv. <https://arxiv.org/abs/2005.07959>

BIBLIOGRAPHY

- [11] Tsitsulin, A., Mottin, D., Karras, P., Bronstein, A., & Müller, E. (2018). “NetLSD: Hearing the Shape of a Graph.” Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining. <https://doi.org/10.1145/3219819.3219991>
- [12] Li, L., Li, J., Song, Y., Qin, S., Wen, Q., & Gao, F. (2023). Proactively incremental-learning QAOA. arXiv. <https://doi.org/10.48550/arXiv.2311.02302>
- [13] Amosy, O., Danzig, T., Porat, E., Chechik, G., & Makmal, A. (2022). Iterative-Free Quantum Approximate Optimization Algorithm Using Neural Networks. ArXiv. <https://arxiv.org/abs/2208.09888>
- [14] Jain, N., Coyle, B., Kashefi, E., & Kumar, N. (2021). Graph neural network initialisation of quantum approximate optimisation. ArXiv. <https://doi.org/10.22331/q-2022-11-17-861>
- [15] Wang, H., Li, P., Chen, M., Cheng, J., Liu, J., & Chen, T. (2024). GroverGPT: A Large Language Model with 8 Billion Parameters for Quantum Searching. ArXiv. <https://arxiv.org/abs/2501.00135>
- [16] Nakaji, K., Kristensen, L. B., Kemmoku, R., Campos-Gonzalez-Angulo, J. A., Vakili, M. G., Huang, H., Bagherimehrab, M., Gorgulla, C., Wong, F., McCaskey, A., Kim, J. S., Nguyen, T., Rao, P., Gao, Q., Sugawara, M., Yamamoto, N., & Aspuru-Guzik, A. (2024). The generative quantum eigensolver (GQE) and its application for ground state search. ArXiv. <https://arxiv.org/abs/2401.09253>
- [17] Farhi, E., Goldstone, J., & Gutmann, S. (2014). A Quantum Approximate Optimization Algorithm. ArXiv. <https://arxiv.org/abs/1411.4028>
- [18] Minaee, S., Mikolov, T., Nikzad, N., Chenaghlu, M., Socher, R., Amatriain, X., & Gao, J. (2024). Large Language Models: A Survey. ArXiv. <https://arxiv.org/abs/2402.06196>
- [19] Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M. A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., Rodriguez, A., Joulin, A., Grave, E., & Lample, G. (2023). LLaMA: Open and Efficient Foundation Language Models. ArXiv. <https://arxiv.org/abs/2302.13971>
- [20] Open Quantum Computing. QAOA. <https://github.com/OpenQuantumComputing/QAOA>, 2026. Accessed: 2026-05-03.
- [21] JuliaGPU. Adapt.jl. <https://github.com/JuliaGPU/Adapt.jl>, 2026. Accessed: 2026-05-03.

THANK YOU

Appendix



Why Use AI Instead of Running ADAPT-QAOA Directly?

- **ADAPT-QAOA** means mapping from Graph $\rightarrow$ Quantum Circuit. This is AI's advantage
- **Amortized Optimization:** Shift computation from inference time to training time.
- **QAOA** circuits has structure, it's not RANDOM

Quantum Gates

1. SINGLE-QUBIT GATES

Operate on one qubit. Change the state or phase.

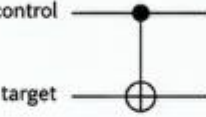
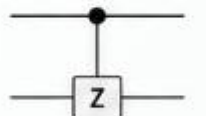
GATE	FUNCTION	WHAT IT DOES	HOW (CIRCUIT)
X (Pauli-X)	Bit flip	Like NOT gate $ 0\rangle \leftrightarrow 1\rangle$	 $\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$
Y (Pauli-Y)	Bit + phase flip	Flips bit and adds a phase (Y-rotation)	 $\begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}$
Z (Pauli-Z)	Phase flip	Changes phase of $ 1\rangle$: $ 1\rangle \rightarrow - 1\rangle$	 $\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$
H (Hadamard)	Superposition	Creates equal superposition $\frac{1}{\sqrt{2}}(0\rangle + 1\rangle)$	 $\frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$



These gates are the basic operations on a single qubit.

2. MULTI-QUBIT / ENTANGLING GATES

Operate on two or more qubits. Create correlations (entanglement).

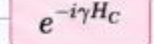
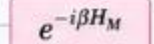
GATE	FUNCTION	WHAT IT DOES	HOW (CIRCUIT)
CNOT (CX)	Controlled bit flip	Flips target qubit when control qubit is 1. $ 00\rangle \rightarrow 00\rangle$ $ 10\rangle \rightarrow 11\rangle$ $ 01\rangle \rightarrow 01\rangle$ $ 11\rangle \rightarrow 10\rangle$	
CZ	Controlled phase flip	Applies a -1 phase when both qubits are in $ 1\rangle$. $ 11\rangle \rightarrow - 11\rangle$	
CY	Controlled Y operation	Applies Y gate to target qubit when control qubit is 1.	



These gates entangle qubits and enable quantum advantage.

3. PARAMETERIZED GATES

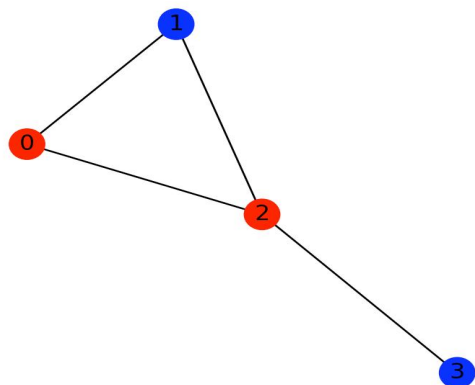
Have learnable parameters. Key to QAOA & ADAPT-QAOA.

GATE	FUNCTION	ROLE IN QAOA / ADAPT-QAOA	HOW (CIRCUIT)
$R_x(\theta)$	Rotate around X-axis	Learnable rotation. Used in mixer operations and variational ansätze.	 $\begin{bmatrix} \cos \frac{\theta}{2} & -i \sin \frac{\theta}{2} \\ -i \sin \frac{\theta}{2} & \cos \frac{\theta}{2} \end{bmatrix}$
$R_y(\theta)$	Rotate around Y-axis	Learnable rotation. Used for state preparation and adaptive ansatz.	 $\begin{bmatrix} \cos \frac{\theta}{2} & -\sin \frac{\theta}{2} \\ \sin \frac{\theta}{2} & \cos \frac{\theta}{2} \end{bmatrix}$
$R_z(\theta)$	Rotate around Z-axis	Learnable phase rotation. Used in cost-dependent evolutions.	 $\begin{bmatrix} e^{-i\theta/2} & 0 \\ 0 & e^{i\theta/2} \end{bmatrix}$
$e^{-i\gamma H_C}$	Cost evolution	Encodes the problem objective (problem Hamiltonian H_C).	 $e^{-i\gamma H_C}$
$e^{-i\beta H_M}$	Mixer evolution	Explores the solution space by mixing quantum states (mixer Hamiltonian H_M).	 $e^{-i\beta H_M}$

Quantum Operators

	CLASSICAL (2 bits)	QUANTUM (2 qubits)																				
1 Mathematical Operation / Operator (What it does)	SWAP operation on 2 bits <table><tr><th>Input (A B)</th><th>Output (B A)</th></tr><tr><td>00</td><td>00</td></tr><tr><td>01</td><td>10</td></tr><tr><td>10</td><td>01</td></tr><tr><td>11</td><td>11</td></tr></table> A B A B Swaps the two bits.	Input (A B)	Output (B A)	00	00	01	10	10	01	11	11	X_0X_1 operator = $X \otimes X$ <table><tr><th>Input</th><th>Output</th></tr><tr><td>$00\rangle$</td><td>$11\rangle$</td></tr><tr><td>$01\rangle$</td><td>$10\rangle$</td></tr><tr><td>$10\rangle$</td><td>$01\rangle$</td></tr><tr><td>$11\rangle$</td><td>$00\rangle$</td></tr></table> q0 X q0 q1 X q1 Flips both qubits. $X_0X_1 = X \otimes X$	Input	Output	$ 00\rangle$	$ 11\rangle$	$ 01\rangle$	$ 10\rangle$	$ 10\rangle$	$ 01\rangle$	$ 11\rangle$	$ 00\rangle$
Input (A B)	Output (B A)																					
00	00																					
01	10																					
10	01																					
11	11																					
Input	Output																					
$ 00\rangle$	$ 11\rangle$																					
$ 01\rangle$	$ 10\rangle$																					
$ 10\rangle$	$ 01\rangle$																					
$ 11\rangle$	$ 00\rangle$																					
2 Gate / Circuit (How we implement it)	SWAP gate / circuit Implemented with 3 CNOT gates A B A B Effect: swaps the two bits.	$e^{-i\beta X_0X_1}$ gate q0 $e^{-i\beta X_0X_1}$ q0 q1 q1 Unitary generated by the operator X_0X_1. Creates entanglement for most β.																				
3 Physical Implementation (What actually happens)	Transistors changing voltages A B A B Electrical signals (0V or 1V) move through transistors to realize the SWAP gate.	Quantum hardware pulses Microwave pulses Control of qubit states Evolution of amplitudes & phases Quantum chip executes the gate Physical systems (superconducting qubits, ions, photons, etc.) evolve according to the unitary $e^{-i\beta X_0X_1}$.																				

Quantum Operators



4-node graph

qaoa_double_pool

- Keep only $i < j$ pairs.
- Remove duplicates like X_0Y_1 and X_1Y_0 .
- Treat the pair as an unordered interaction.



$X_0 X_1 X_2 X_3$
 $Y_0 Y_1 Y_2 Y_3$
 $X_0X_1 X_0X_2 X_0X_3 X_1X_2 X_1X_3 X_2X_3$
 $Y_0Y_1 Y_0Y_2 Y_0Y_3 Y_1Y_2 Y_1Y_3 Y_2Y_3$
 $X_0Y_1 X_0Y_2 X_0Y_3 X_1Y_2 X_1Y_3 X_2Y_3$
 $X_0Z_1 X_0Z_2 X_0Z_3 X_1Z_2 X_1Z_3 X_2Z_3$
 $Y_0Z_1 Y_0Z_2 Y_0Z_3 Y_1Z_2 Y_1Z_3 Y_2Z_3$
 $Z_0Z_1 Z_0Z_2 Z_0Z_3 Z_1Z_2 Z_1Z_3 Z_2Z_3$



45 Operators

Unitary & Hamiltonian

Unitary operator $U = e^{-it\hat{H}}$

$\hat{H}$ (Hamiltonian) represents the energy of the quantum system.

Example: a tiny magnet in an external magnetic field B along the z-direction

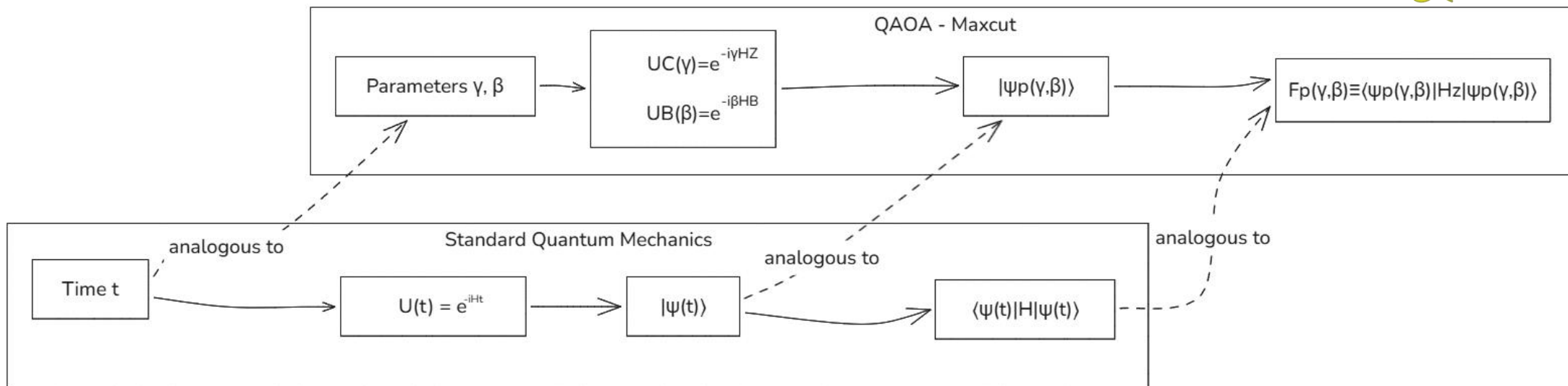
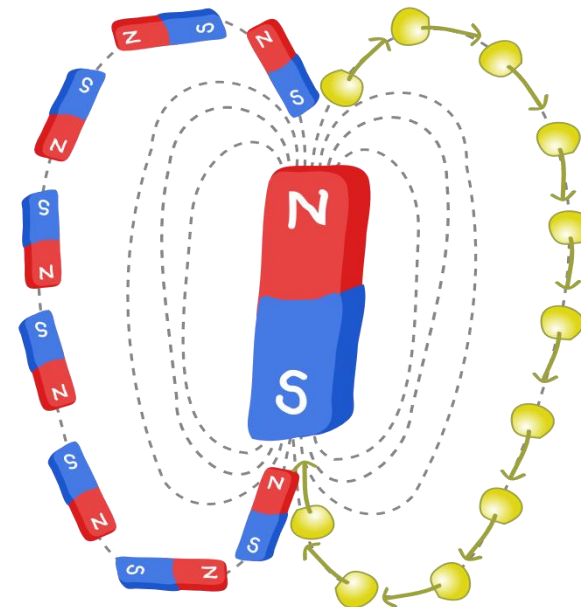
$$\hat{H} = -\frac{eB}{2m_e} Z = -aZ$$

Z: Pauli-Z operator

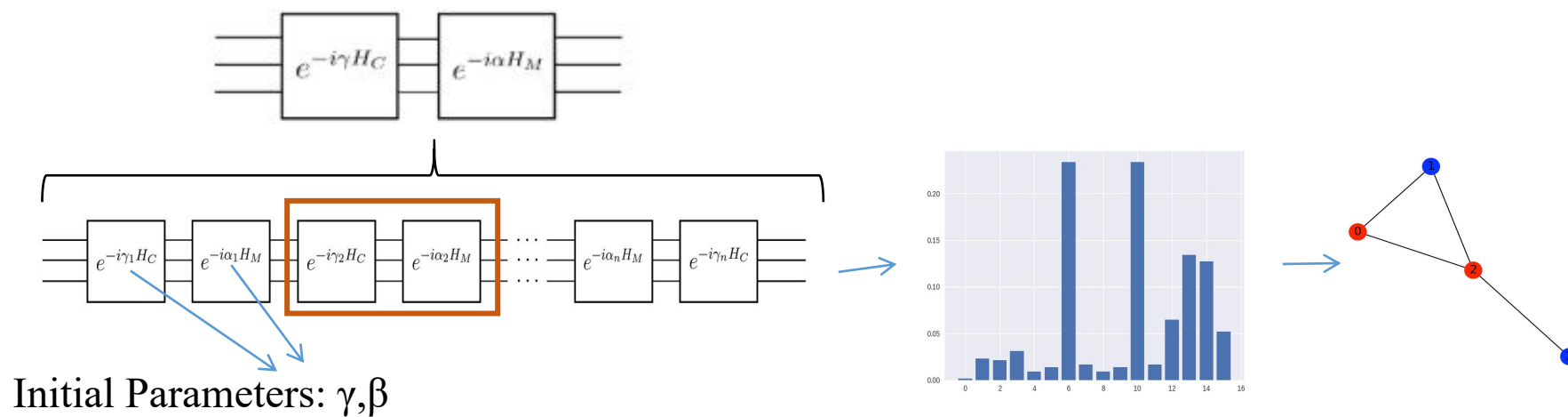
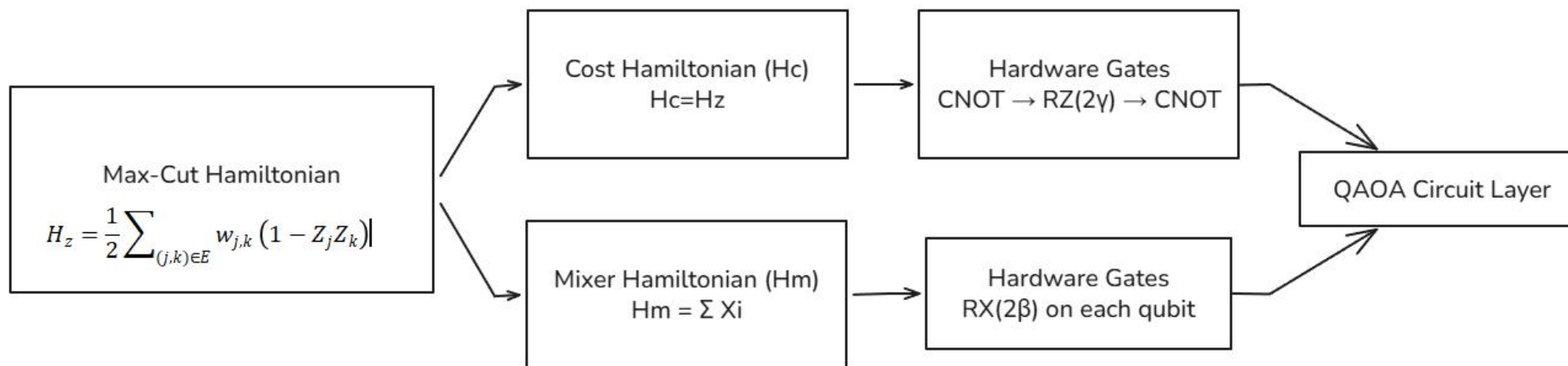
e: charge of the electron

m: : mass of the electron

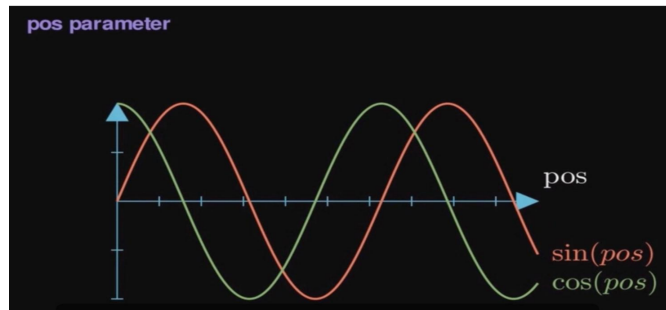
B: strength of the external magnetic field



QAOA circuit on Hardware



Positional Encoding



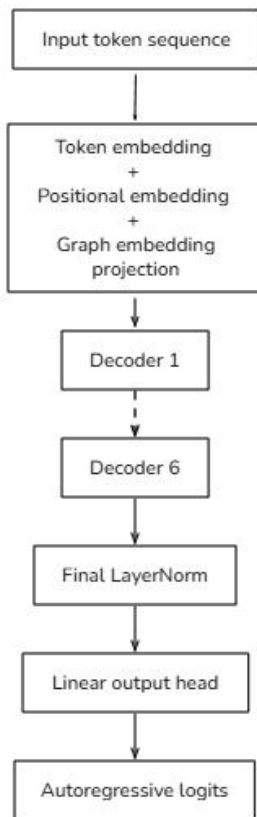
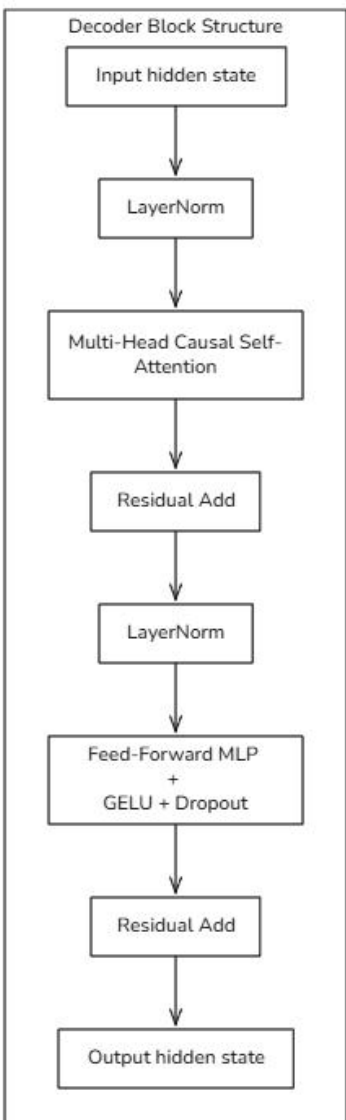
Cat eats Fish



Fish eats Cat

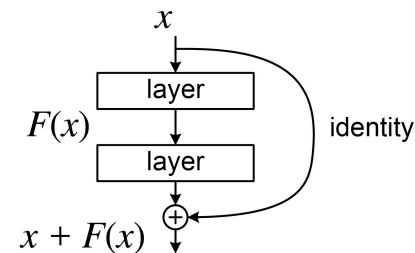
LayerNorm:

- Stabilizes training
- Keeps values in a healthy range
- Makes gradients flow better



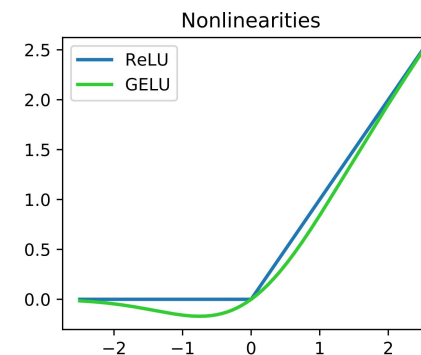
Residual Connection

- Prevents information loss
- Helps very deep networks train



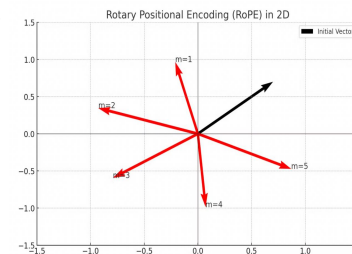
GeLU

- Less harsh cutoff
- Smoother gradients
- Better than ReLU for Transformers



Rotary Positional Embedding (RoPE)

- Modifies attention mathematically.
not add to input embedding
- Scales better to long context
- Relative pos of token: how far?

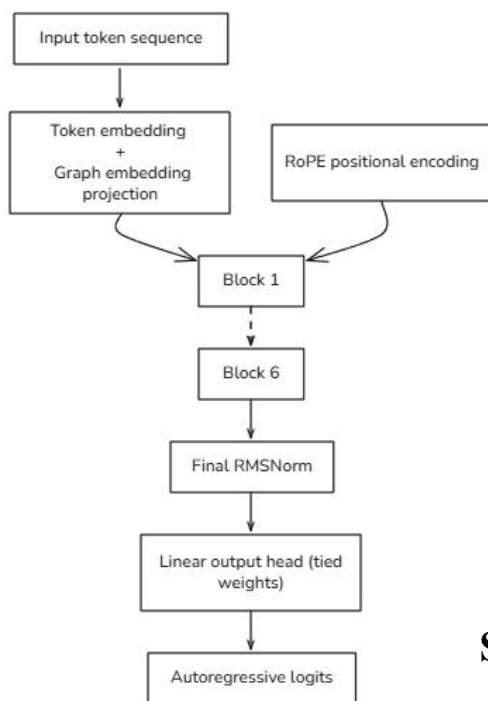
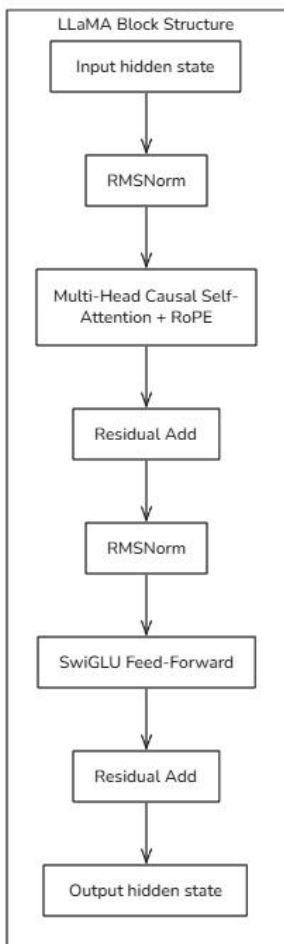
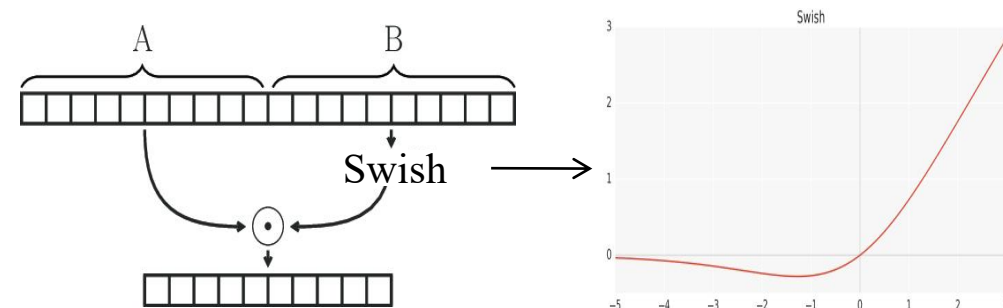


RMSNorm:

- RMSNorm does NOT subtract the mean
- It only scales by magnitude
- Faster, simpler, less computation

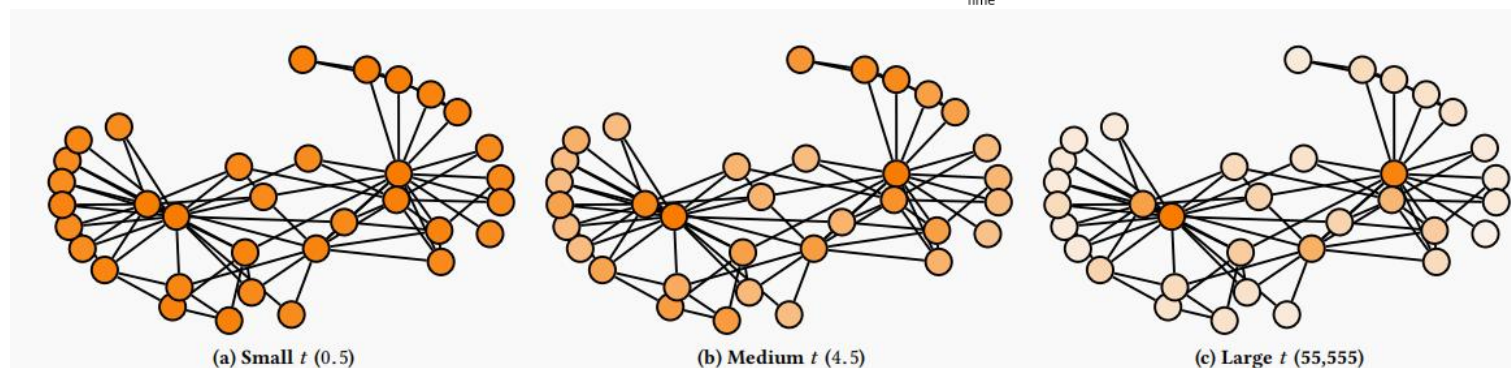
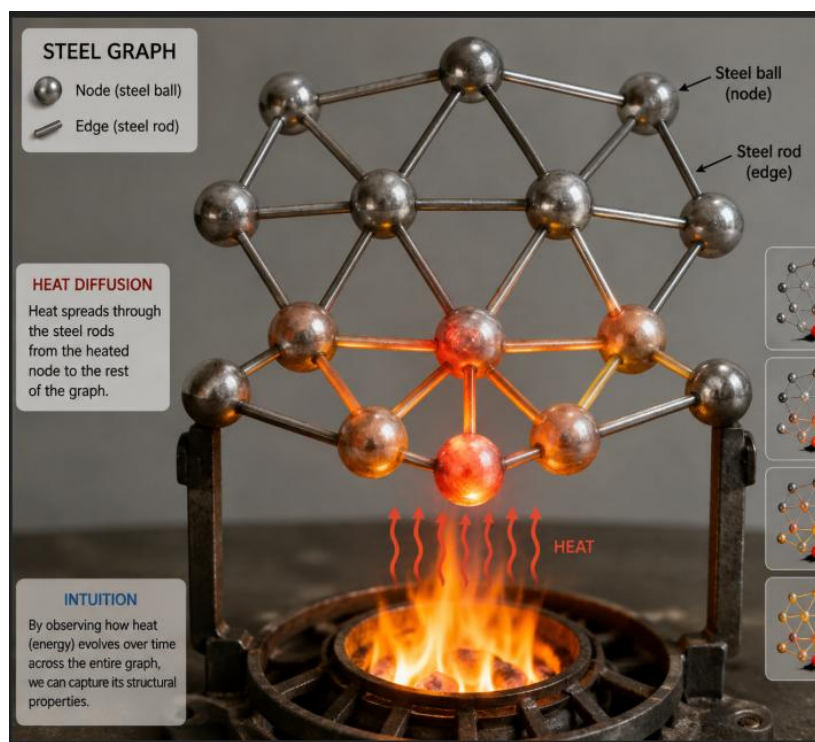
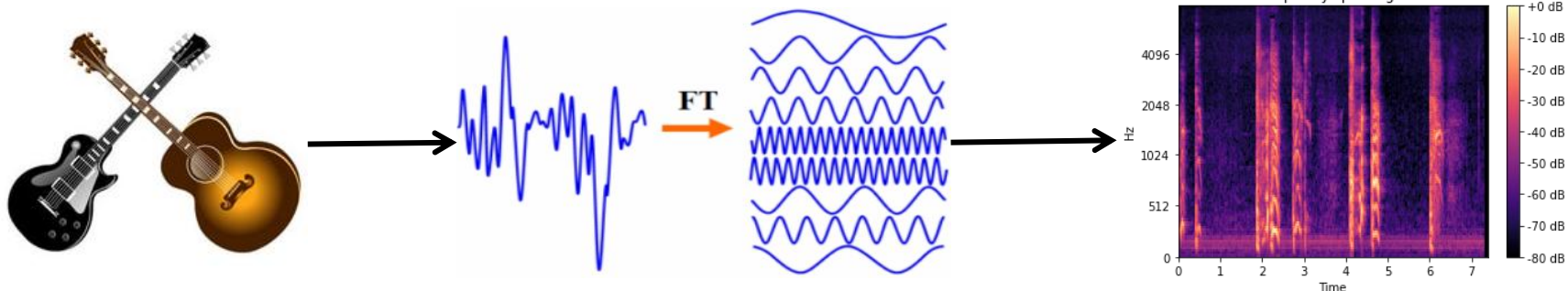
SwiGLU

- Choose important features
- Suppress useless ones
- Selective information flow



NetLSD

Do they sound alike?



- Permutation and size-invariant
- Scale-adaptive
- Efficiently computable graph representation

NetLSD

1. The Graph (4 nodes)



A simple path: $A - B - C - D$

2. Graph Laplacian $L = D - A$

Adjacency matrix A

$$\begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

Degree matrix D

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 \\ 0 & 0 & 2 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Laplacian $L = D - A$

$$\begin{bmatrix} 1 & -1 & 0 & 0 \\ -1 & 2 & -1 & 0 \\ 0 & -1 & 2 & -1 \\ 0 & 0 & -1 & 1 \end{bmatrix}$$

3. Eigenvalues of L (the frequencies)

Solve $Lv_i = \lambda_i v_i$

Eigenvalues (sorted):

$$\begin{aligned} \lambda_1 &= 0 \\ \lambda_2 &\approx 0.586 \\ \lambda_3 &= 2 \\ \lambda_4 &\approx 3.414 \end{aligned}$$

These are the graph frequencies (like vibration frequencies).

4. NetLSD: Heat trace $h(t) = \sum_t e^{-t\lambda_i}$

Intuition

At time t , each frequency λ_i contributes $e^{-t\lambda_i}$ to the total remaining heat.

Small λ_i : slow decay

Large λ_i : fast decay

Contribution of each eigenvalue

$\lambda_1 = 0$ (constant mode)

$\lambda_2 = 0.586$

$\lambda_3 = 2$

$\lambda_4 = 3.414$

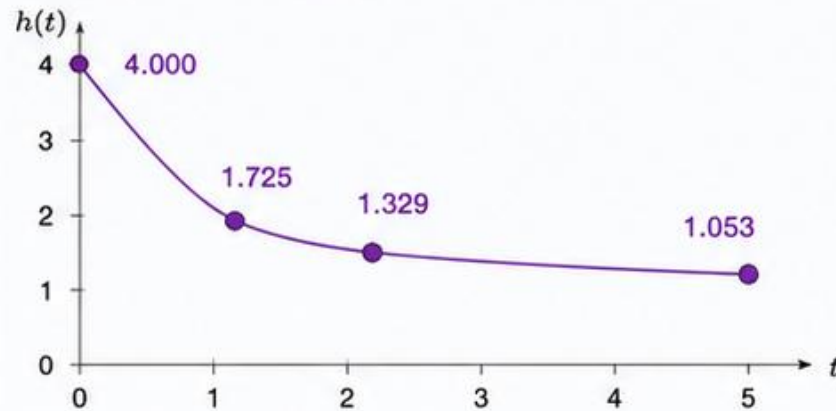
Sum = $h(t)$

t	0	1	2	5
$e^{-0 \cdot t} = 1$	$e^{-0 \cdot t} = 1$	$e^{-0 \cdot t} = 1$	$e^{-0 \cdot t} = 1$	$e^{-0 \cdot t} = 1$
1	$e^{-0.586} \approx 0.557$	$e^{-1.172} \approx 0.310$	$e^{-2.93} \approx 0.053$	
1	$e^{-2} \approx 0.135$	$e^{-4} \approx 0.018$	$e^{-10} \approx 0.00005$	
1	$e^{-3.414} \approx 0.033$	$e^{-6.828} \approx 0.001$	$e^{-17.07} \approx 0$	
	4.000	1.725	1.329	1.053

Add all contributions to get the total remaining heat $h(t)$ at time t .

5. Heat trace curve $h(t)$

$h(t)$ decreases over time as the heat diffuses and dissipates.



6. NetLSD Embedding (Signature)

Sample $h(t)$ at multiple time scales (logarithmically spaced in practice).

Example time scales

$$t = [0, 1, 2, 5]$$

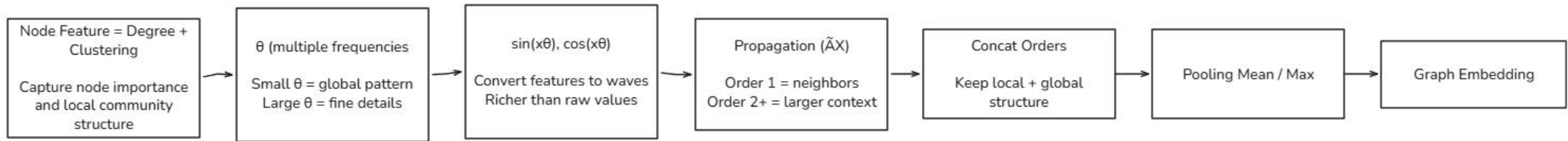


NetLSD embedding (heat trace vector)

$$[4.000, 1.725, 1.329, 1.053]$$

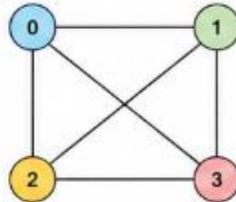
✦ This vector is a compact, multi-scale, permutation-invariant representation of the graph.

FEATHER



FEATHER

1. Example Graph (4 nodes)



Edges:

(0-1), (1-3), (3-2), (2-0), (0-3), (1-2)

2. Compute node features

Node	Degree	$\log(\text{degree}+1)$	Clustering coefficient
0	3	1.386	1.00
1	3	1.386	1.00
2	3	1.386	1.00
3	3	1.386	1.00

- Degree = 3 for all nodes.
- Each node's neighbors are connected to each other (triangle), so clustering coefficient = 1.

Feature matrix $X \in \mathbb{R}^{4 \times 2}$

$$X = \begin{bmatrix} 1.386 & 1.000 \\ 1.386 & 1.000 \\ 1.386 & 1.000 \\ 1.386 & 1.000 \end{bmatrix}$$

3. Choose theta and compute Fourier features (sin & cos)

Let $\text{eval_points} = 3$, $\text{theta_max} = 2.5$

θ (evaluation points) = [0.5, 1.5, 2.5]

For each feature value x and each θ , compute $\cos(x\theta)$ and $\sin(x\theta)$.

Example: for $x = 1.386$ (log degree+1)

θ	$\cos(x\theta)$			$\sin(x\theta)$		
0.5	0.769	-0.481	-0.943	0.639	0.877	0.333

Example: for $x = 1.000$ (clustering coef)

θ	$\cos(x\theta)$			$\sin(x\theta)$		
2.5	0.878	0.071	-0.801	0.479	0.997	0.598

Per node dimension after sin & cos:
2 features $\times$ 3 θ $\times$ 2 (sin & cos) = 12

Why theta and sin/cos?

- Theta (frequencies):** different θ values look at the feature from different "frequencies" (scales). Small $\theta \rightarrow$ smooth pattern, large $\theta \rightarrow$ capture fine details.
- Sin & cos:** together they form a complete, non-redundant representation of each frequency (like waves). This preserves information and makes the embedding expressive.

Intuition: we turn node features into waves at multiple frequencies.

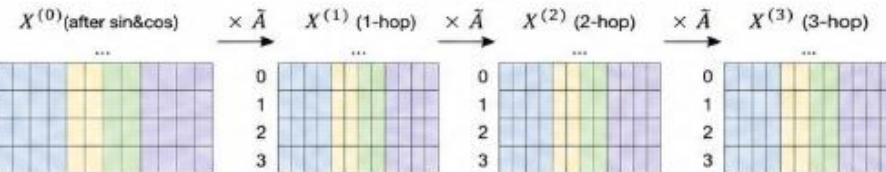
4. Propagate through the graph (use normalized adjacency $\tilde{A}$)

All degrees = 3 $\rightarrow D^{-1} = \text{diag}(1/3, 1/3, 1/3, 1/3)$

Adjacency A

$$A = \begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix}$$

$$\tilde{A} = D^{-1}A = \frac{1}{3} \begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix}$$



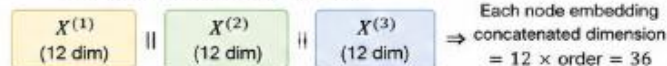
Each multiplication averages and mixes neighbors' features.
Higher order = larger neighborhood you can "see".
(Here the graph is symmetric, so rows stay similar.)

Why order?

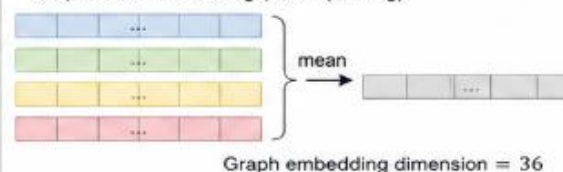
- Order = how many hops information travels.
- Order 1: immediate neighbors.
- Order 2: neighbors of neighbors.
- Larger order captures wider structural context.

6. Final embedding & pooling (graph-level)

Concatenate multi-scale features



Graph-level embedding (mean pooling)



Summary of dimensions

#features = 2, eval_points = 3, order = 3

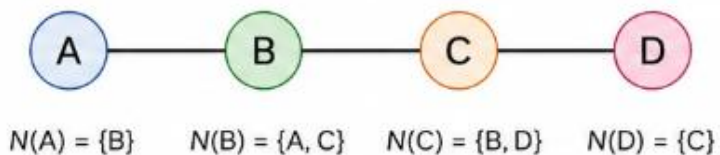
- After sin & cos per node: $2 \times 3 \times 2 = 12$
- Final node embedding: $12 \times 3 = 36$
- Graph embedding (after pooling): 36

In code example: $\text{eval_points} = 25$, $\text{order} = 5$
 $\Rightarrow 2 \times 25 \times 2 \times 5 = 500$

GNN architecture



Example Graph (4 nodes)



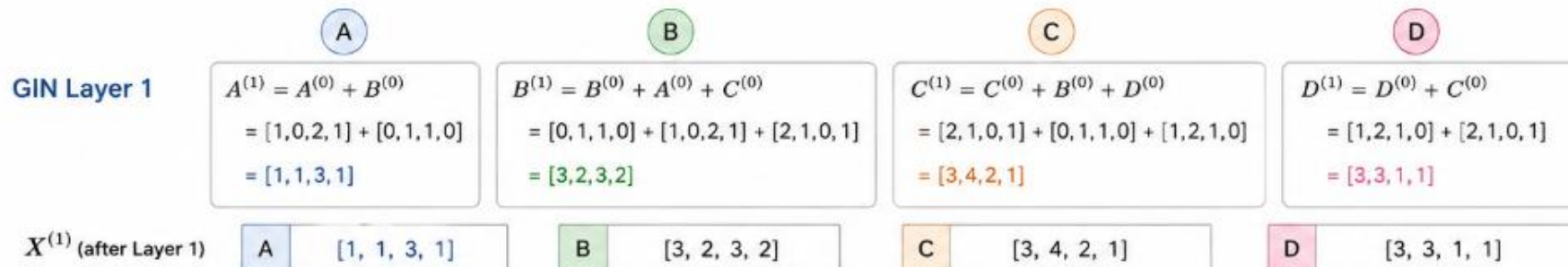
Initial Node Features $X^{(0)}$ ($n \times 4$)

	f_1	f_2	f_3	f_4
A	1	0	2	1
B	0	1	1	0
C	2	1	0	1
D	1	2	1	0

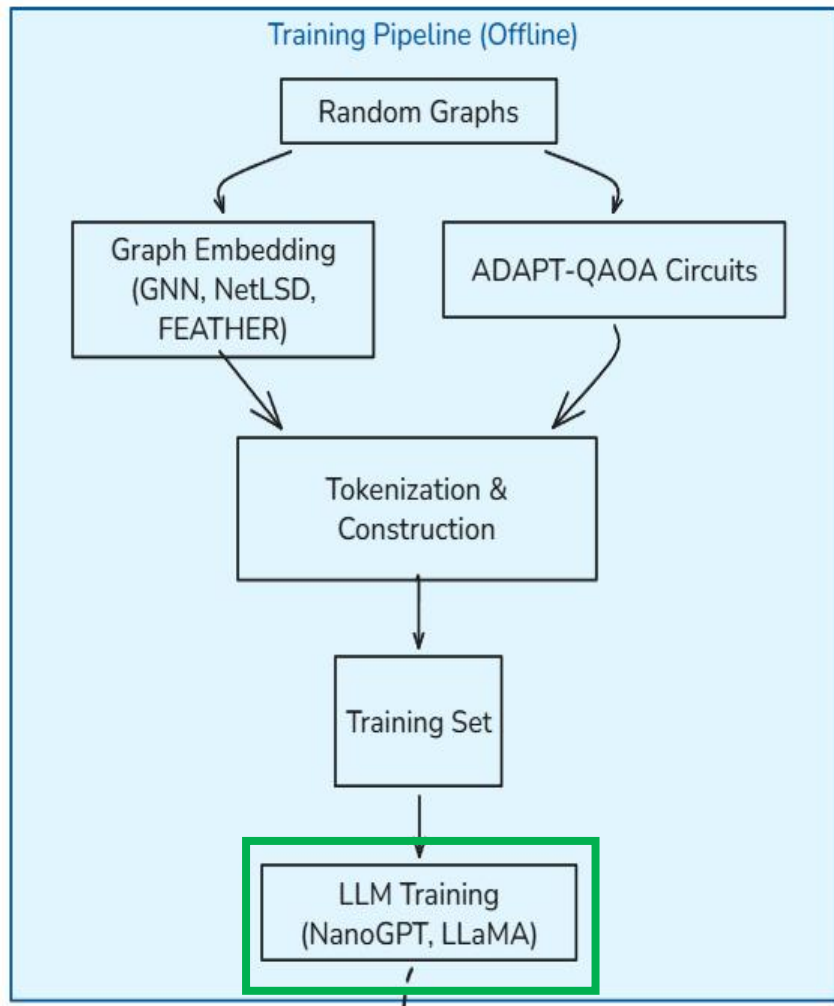
GIN Update Rule

$$h_v^{(k)} = \text{MLP}^{(k)} \left((1 + \epsilon) \cdot h_v^{(k-1)} + \sum_{u \in N(v)} h_u^{(k-1)} \right)$$

($\epsilon = 0$ in this example)



LLM Training

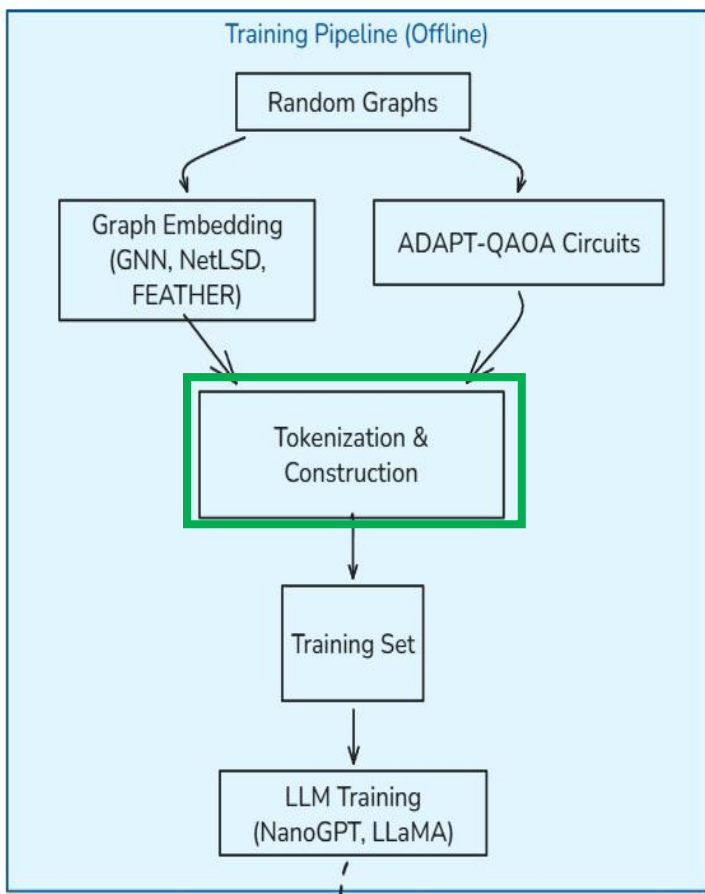


- Autoregressive next-token prediction loss

$$U = \{u_1, u_2, \dots, u_n\} \xrightarrow{\text{Maximize}} \mathcal{L}_{LM}(U) = \sum_{i=1}^n \log P(u_i | u_1, u_2, \dots, u_{i-1}; \theta)$$

- Pad sequences for batching but ignore padding tokens in loss
- Use ground-truth previous token to stabilize and speed up training

Tokenization & Construction



Preserve Instance Integrity

- No mixing between different graphs or circuits
- Each sequence is self-contained
- Model never sees cross-instance tokens in one context window

Sequence Splitting (for Transformer)

- Random window size:

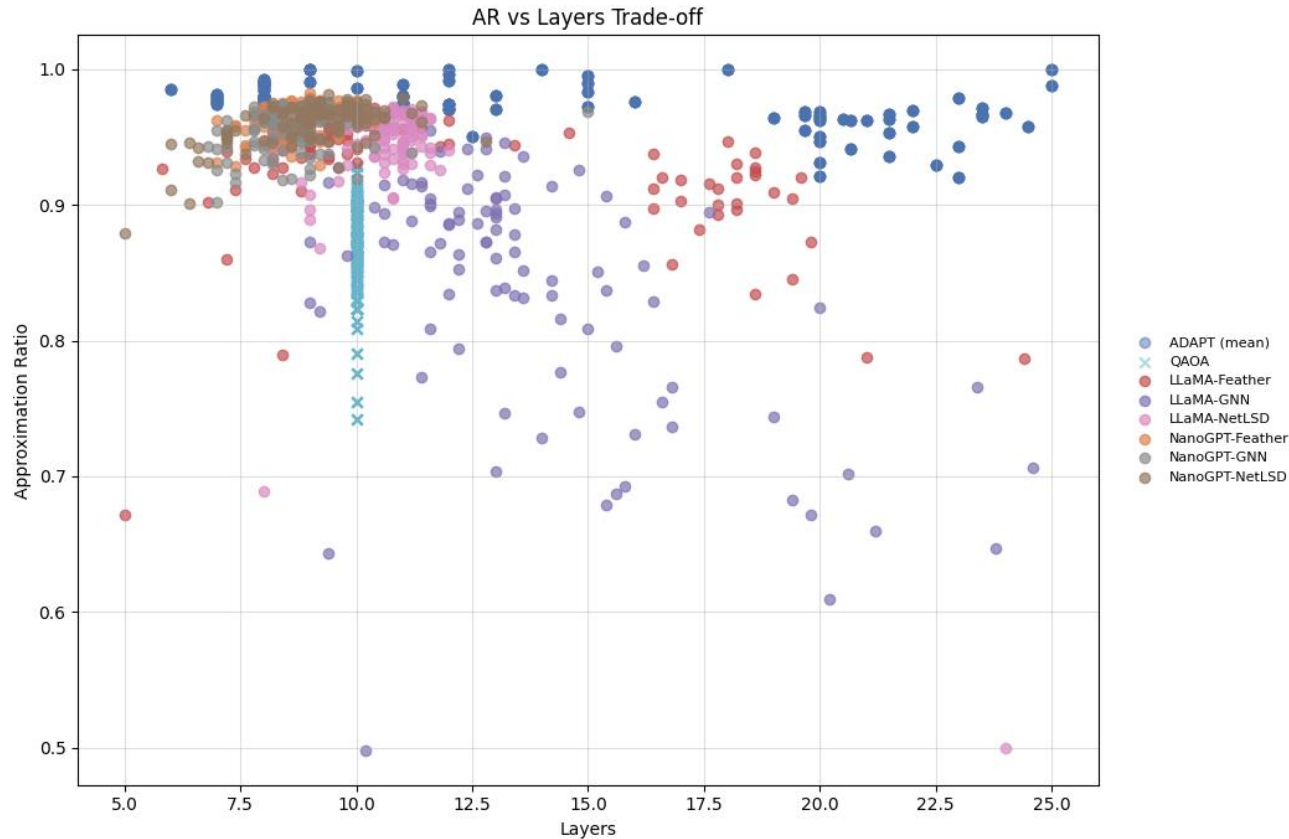
$$b_i \sim \text{Uniform}(b_{\min}, b_{\max}) \quad b_{\max} \leq T$$

- Apply sliding window to generate subsequences
- Convert into (input, target) training pairs

Fixed-Length Input

- Each subsequence is right-padded to length T
- Use attention masks during batching

AR & Layers Trade-off Analysis



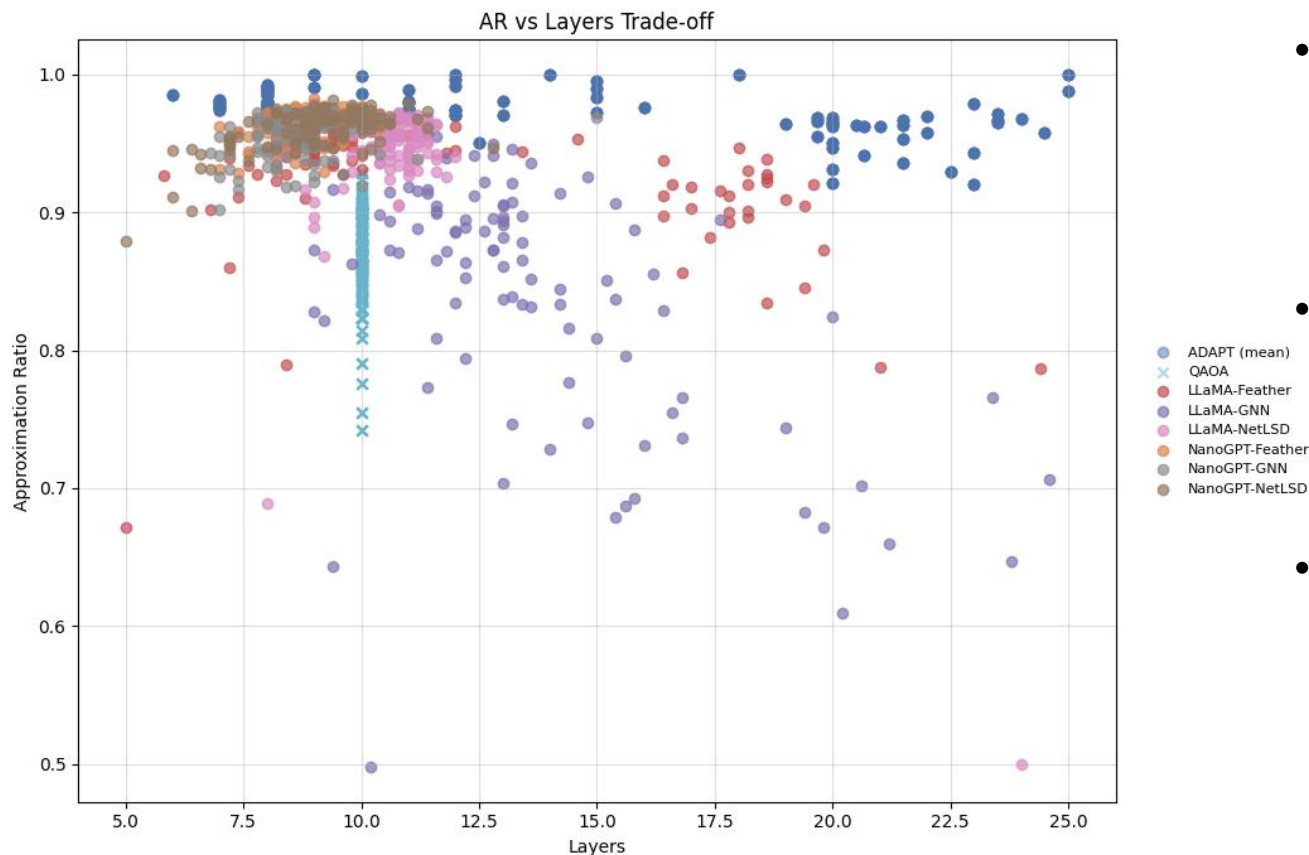
Best quality–efficiency balance: NanoGPT (esp. Feather) achieves high AR (> 0.95) with low depth ($\sim 8\text{--}10$ layers) $\rightarrow$ ideal trade-off

Robust across embeddings: NanoGPT variants stay consistently efficient + high-performing, not sensitive to embedding choice

LLaMA inefficient vs ADAPT trade-off: LLaMA needs deeper circuits (up to ~ 25 layers) with unstable AR, while NanoGPT reaches near-ADAPT performance at much lower depth

AR & Layers Trade Off comparison for 10-node graphs

AR & Layers Trade-off Analysis



AR & Layers Trade Off comparison for 10-node graphs

- **Best quality–efficiency balance:** NanoGPT (esp. Feather) achieves high AR (> 0.95) with low depth ($\sim 8\text{--}10$ layers) $\rightarrow$ ideal trade-off
- **Robust across embeddings:** NanoGPT variants stay consistently efficient + high-performing, not sensitive to embedding choice
- **LLaMA inefficient vs ADAPT trade-off:** LLaMA needs deeper circuits (up to ~ 25 layers) with unstable AR, while NanoGPT reaches near-ADAPT performance at much lower depth